Name: Avinash Rajput

Task Assigned by : Mithun

Task1 : Create an EC2 instance with the user data shared by developers to launch a web application without logging into the server in a particular region.

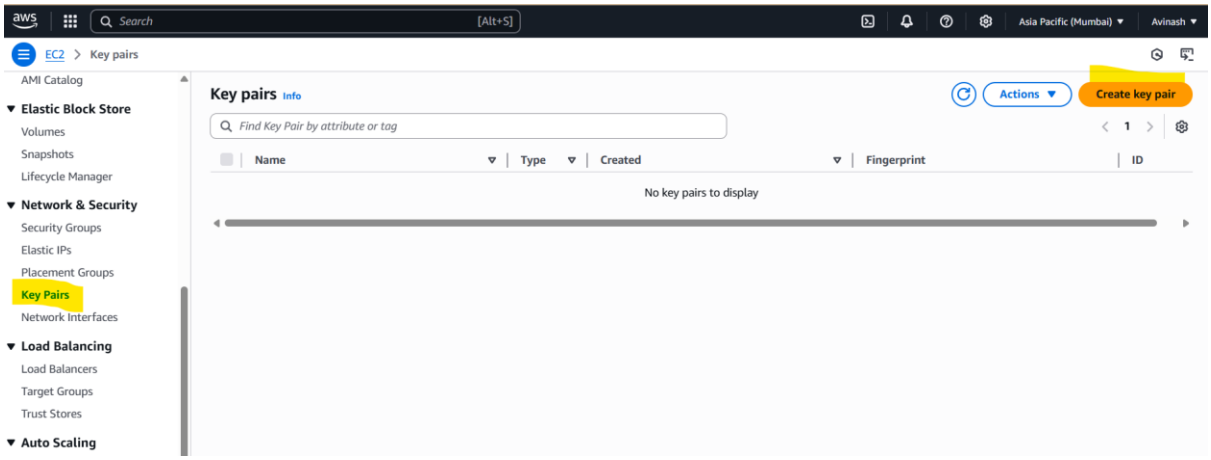
Note: a. Create a new keypair b. Enable HTTP port on network settings to access the web application over the internet.

Step 1: Login to AWS Console

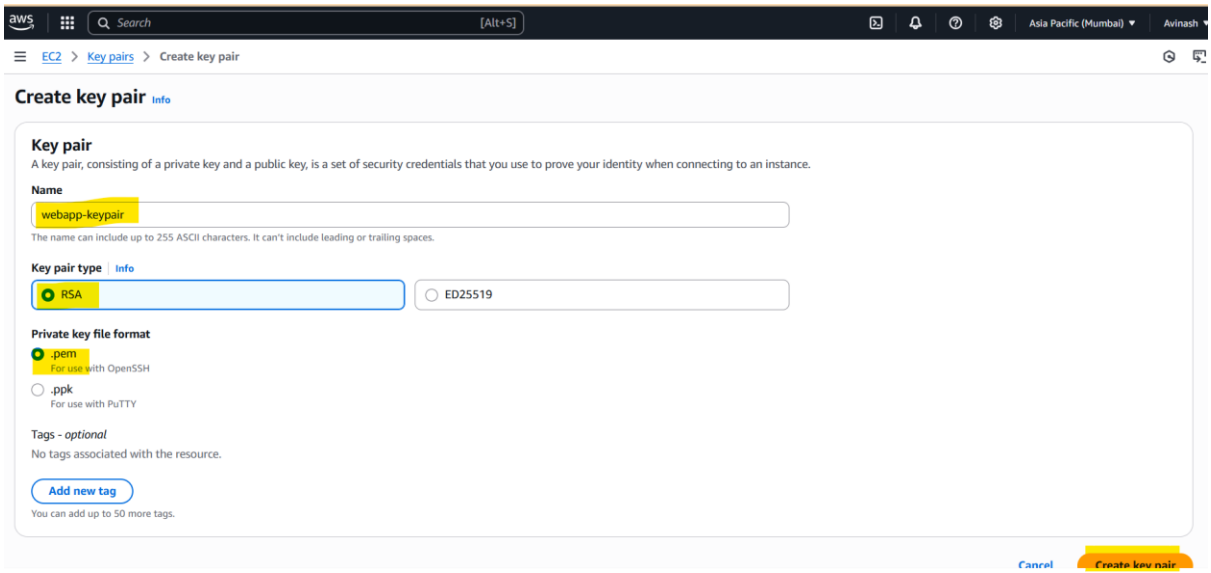
- Go to https://console.aws.amazon.com/ec2

Step 2: Create a Key Pair

- Go to **EC2 > Key Pairs** -> Click **Create key pair ->**

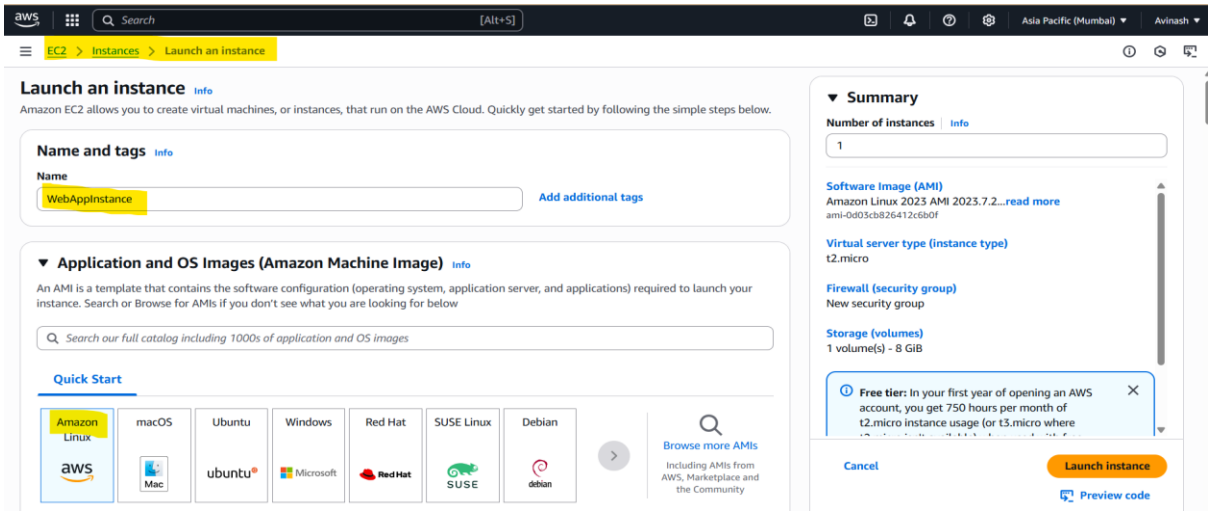


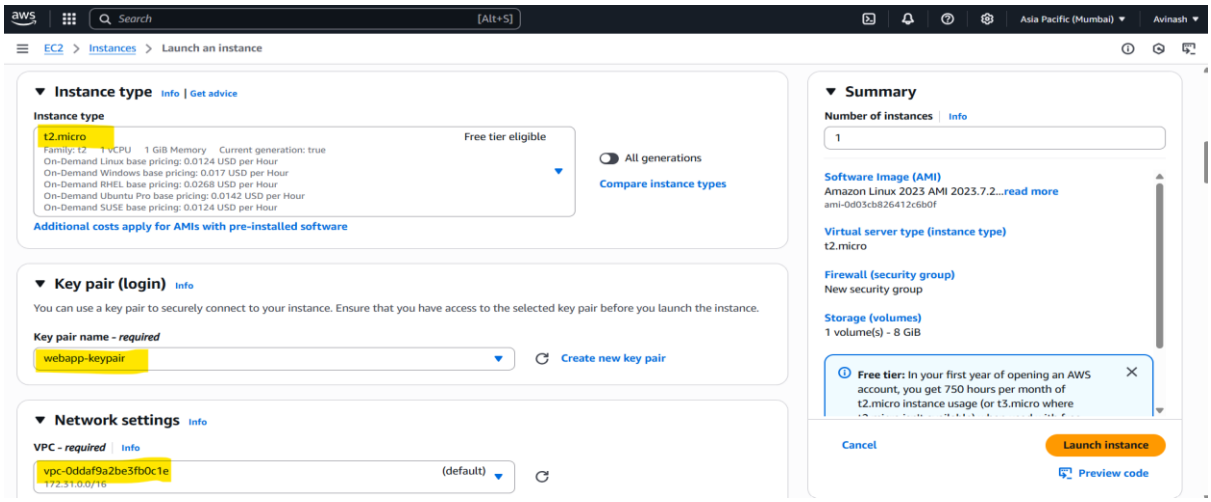
- Click **Create key pair** (name ,key pair type,private key file) & **save the .pem file** securely.

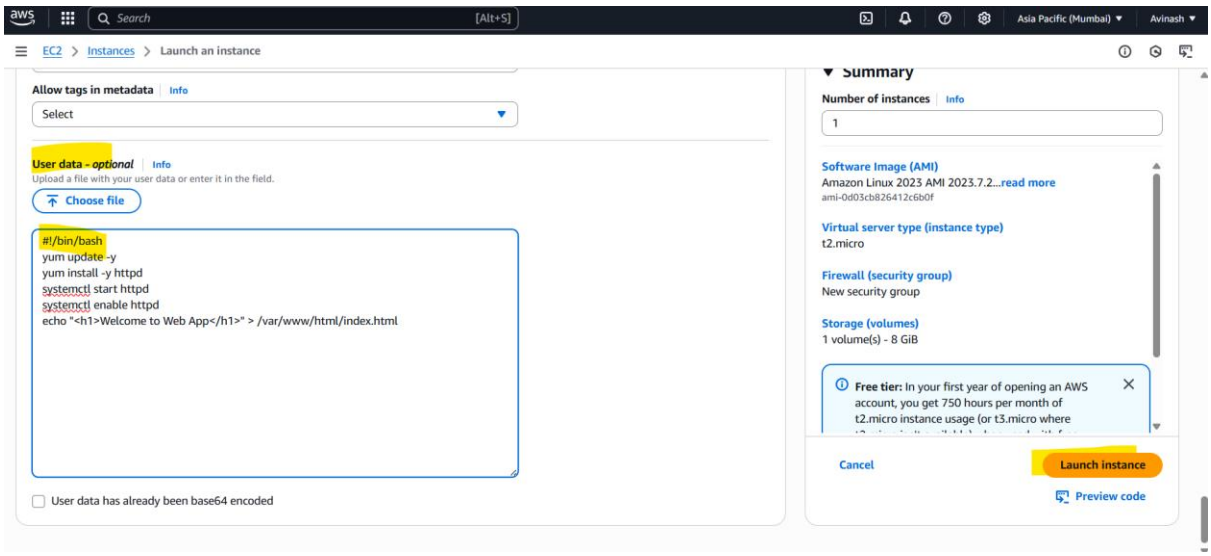


Step 3: Launch EC2 Instance

- Need to insert(Name, AMI. Instance Type, Key pair, Network Settings, User Data)

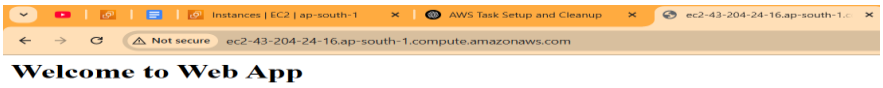






Step 4: Verify Web Application , copy the Public IPv4 DNS.

(http:// ec2-43-204-24-16.ap-south-1.compute.amazonaws.com)



Task 2 : Client Nokia/Samsung/ETC has asked you to create a network design by taking below IP ranges capable of hosting 5000 machines/servers/devices and divide this network into 4 equal parts.

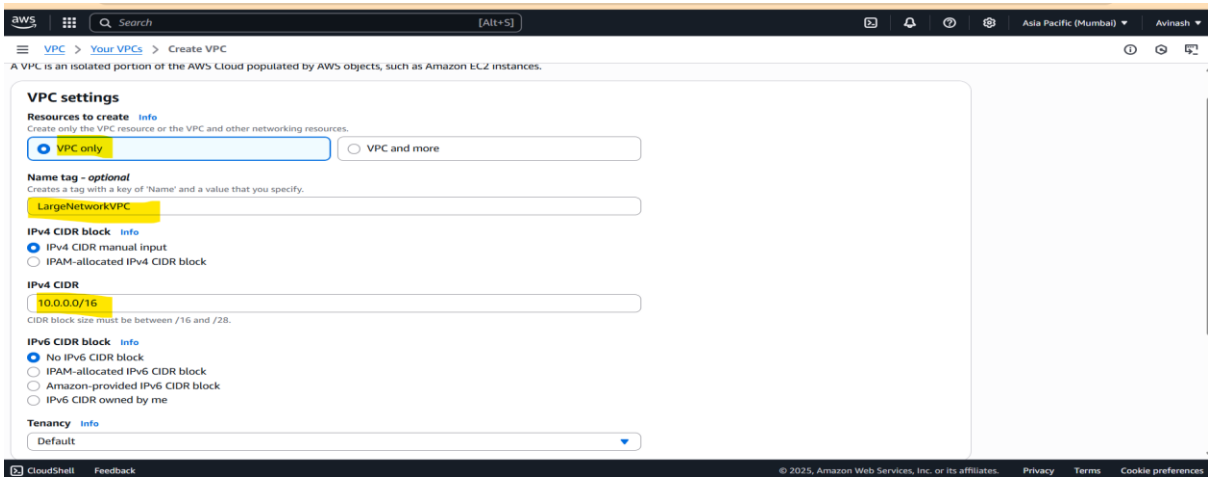
Example: a. 10.0.0.0 b. 172.0.0.0 c. 192.0.0.0

Step 1: Login & Go to VPC Console

- Open: https://console.aws.amazon.com/vpc
- Ensure you're in your desired region e.g: ap-south-1 for Mumbai

🔹 **Step 2: Create a VPC**

1. Click on **"Create VPC"**
2. Choose **VPC Only**
3. Fill in: **Name tag**: LargeNetworkVPC,**IPv4 CIDR block**: 10.0.0.0/16 ,Leave IPv6 blank (unless required),Tenancy: Default
4. Click **Create VPC**

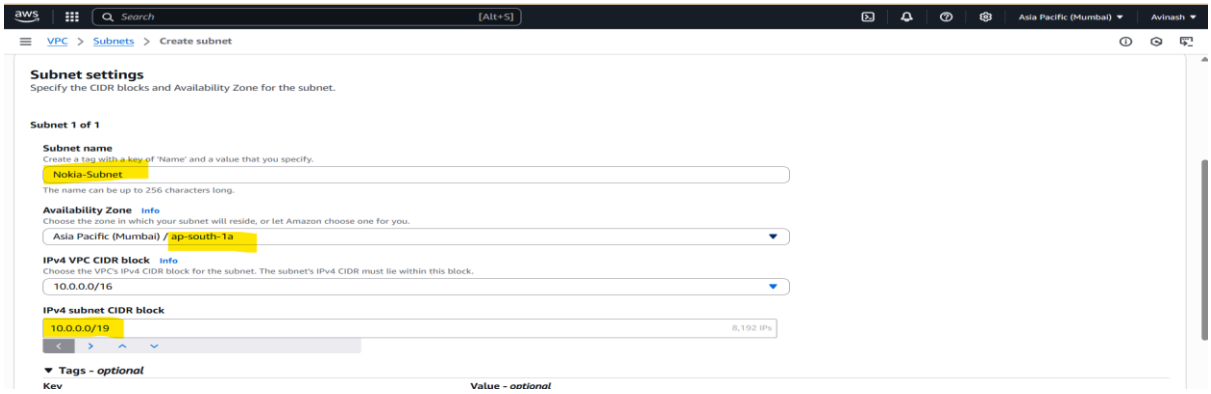


Step 3: Create 4 Subnets (/19 each)

Go to **Subnets > Create Subnet** Fill the following for each:

Subnet 1 (Nokia):

- Name: Nokia-Subnet**,** VPC: LargeNetworkVPC, AZ: Pick any (e.g., ap-south-1a)**,**IPv4 CIDR: 10.0.0.0/19



Subnet 2 (Samsung)

- Name: Samsung-Subnet, VPC: LargeNetworkVPC , AZ: ap-south-1b, IPv4 CIDR: 10.0.32.0/19

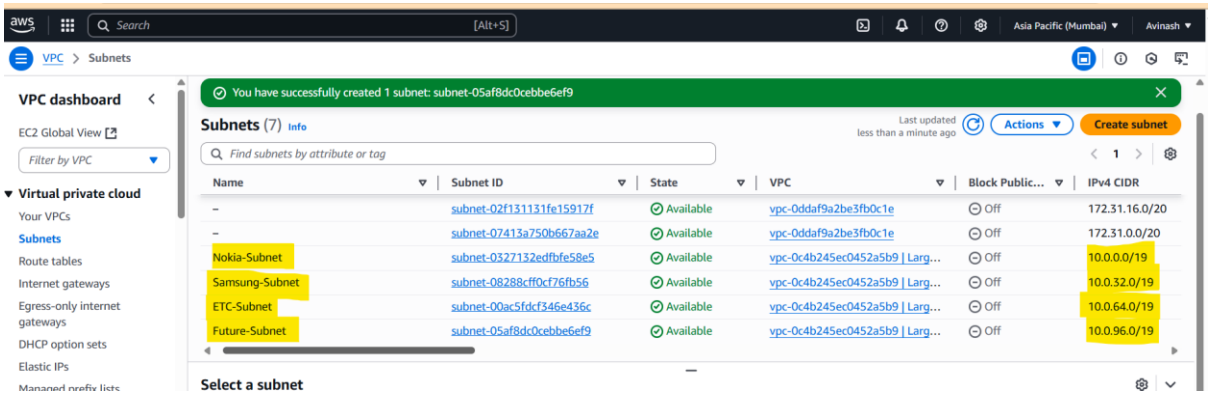
Subnet 3 (ETC)

- Name: ETC-Subnet, VPC: LargeNetworkVPC, AZ: ap-south-1c, IPv4 CIDR: 10.0.64.0/19

Subnet 4 (Reserved/Future)

- Name: Future-Subnet ,VPC: LargeNetworkVPC, AZ: ap-south-1a (or any), IPv4 CIDR: 10.0.96.0/19

Click **Create Subnet**



Task 3 :Convert IP 172.31.38.167 to its binary value.

- **IP Address: 172.31.38.167**

Octet	Decimal	Binary
1st	**172**	**10101100**
2nd	**31**	**00011111**
3rd	**38**	**00100110**
4th	**167**	**10100111**

Answer:172.31.38.167 (decimal)

= 10101100.00011111.00100110.10100111 (binary)

Task 4:

Create a VPC for your client samsung with VPC 192.168.0.0/16

Create 4 subnets equally. Make 2 subnets public and 2 subnets private.

Create a Bastion Host and try to connect to a private instance via this bastion host.

Create a NAT Gateway to provide internet access to the private subnet instances.

Optional but can Try: Configure SQUID proxy server as an alternative to NAT gateway.

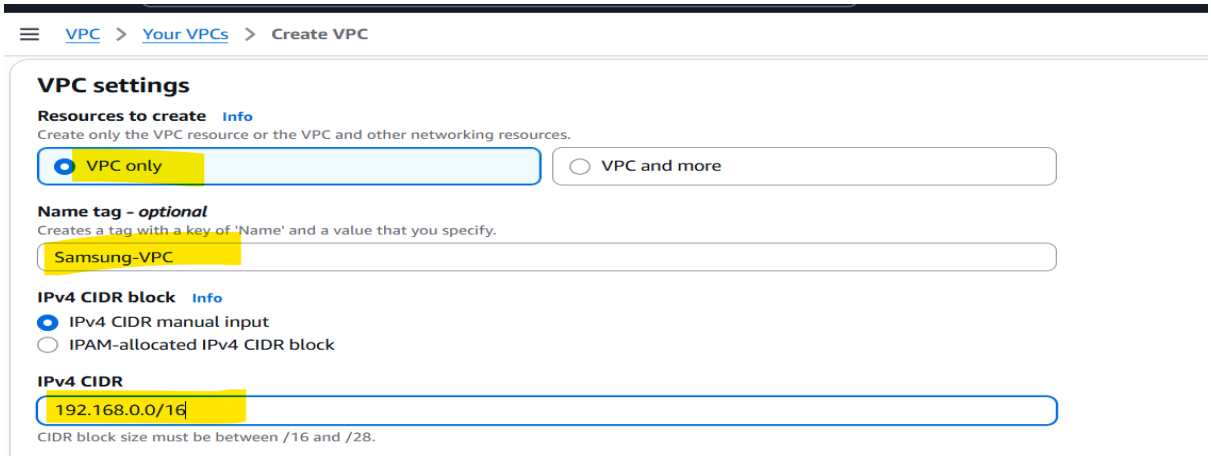
- **Step 1: Create a VPC**
  1. Go to VPC > Create VPC Name: Samsung-VPC ,IPv4 CIDR block: 192.168.0.0/16 ,No IPv6 for now
  2. Click Create

Step 2: Create 4 Subnets

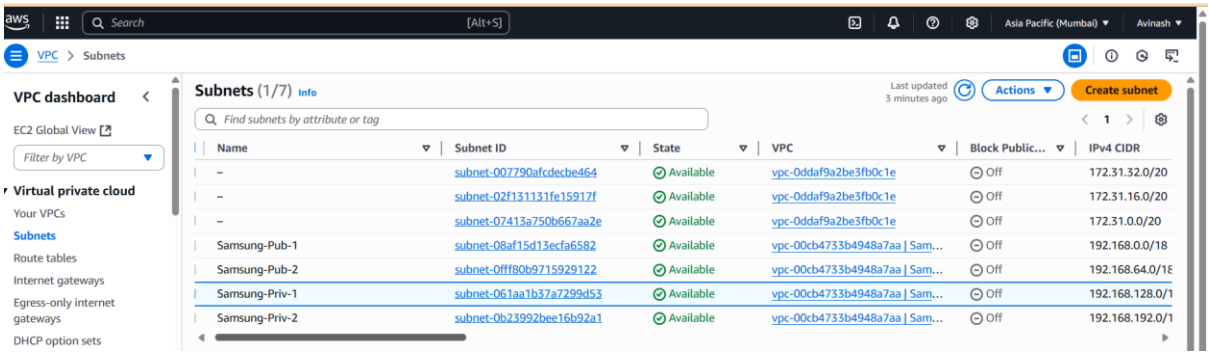
3. You need **4 x /18 subnets** to split 192.168.0.0/16 equally.

Subnet Name	Type	AZ	CIDR
Samsung-Pub-1	Public	ap-south-1a	192.168.0.0/18
Samsung-Pub-2	Public	ap-south-1b	192.168.64.0/18

Subnet Name	Type	AZ	CIDR
Samsung-Priv-1	Private	ap-south-1a	192.168.128.0/18
Samsung-Priv-2	Private	ap-south-1b	192.168.192.0/18

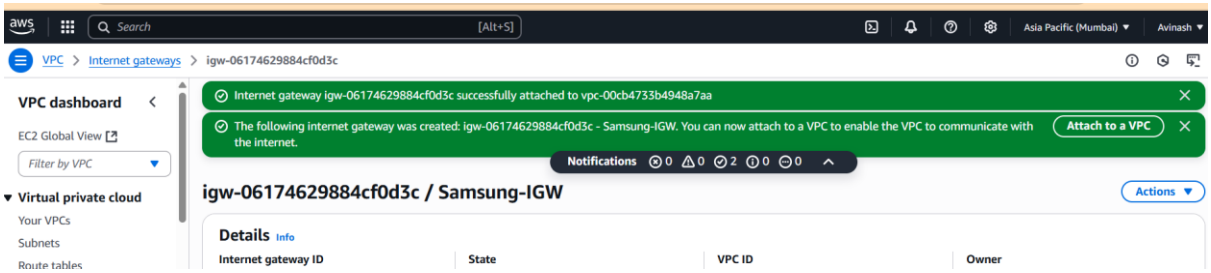


Go to **VPC > Subnets > Create Subnet**, select Samsung-VPC, then create one by one.



Step 3: Create and Attach Internet Gateway

1. Go to **Internet Gatewas -** Create - Name: Samsung-IGW
2. Attach it to Samsung-VPC

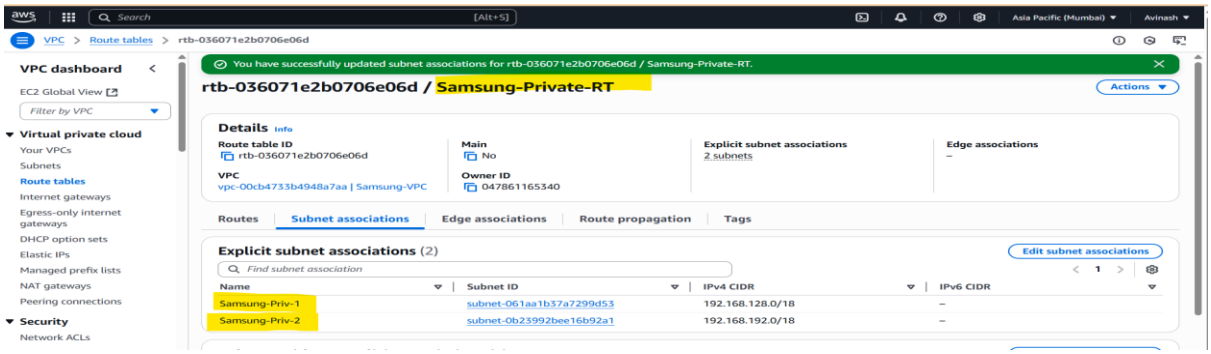


Step 4: Create Route Tables

- **Public Route Table :**Go to **Route Tables > Create Route Table-**Name: Samsung-Public-RT **-**VPC: Samsung-VPC

1. Add Route: Destination: 0.0.0.0/0 ,Target: Samsung-IGW
2. Associate it with: Samsung-Pub-1 and Samsung-Pub-2

- **Private Route Table**
  - Create Samsung-Private-RT
  - Associate it with: Samsung-Priv-1 and Samsung-Priv-2
  - Don’t add a route to 0.0.0.0/0 yet (wait for NAT or proxy)



Step 5: Launch EC2 Instances

Bastion Host (Public)

- Launch EC2 in Samsung-Pub-1
- AMI: Amazon Linux 2
- Key Pair: Create or use existing
- Security Group: Allow **SSH (22)** from your IP
- Name: Bastion-Host

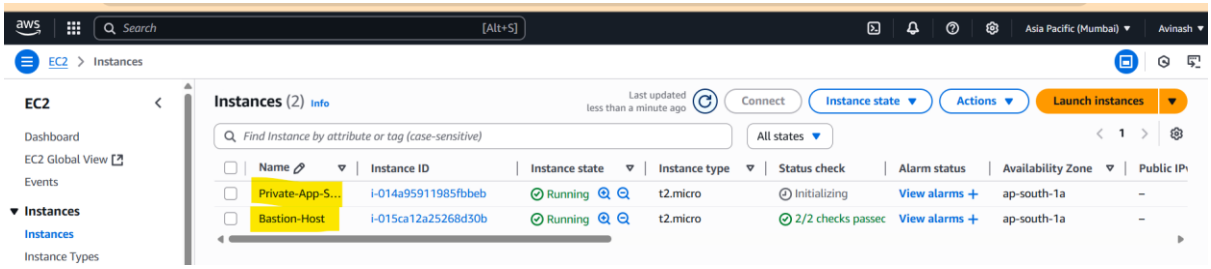
Private Instance

- Launch EC2 in Samsung-Priv-1
- AMI: Amazon Linux 2
- Security Group: Allow **SSH from Bastion Host SG**
- Name: Private-App-Server

4. Launch

- Click **Launch Instance** - You now have:

- **Bastion Host** in public subnet (with internet access)
- **Private EC2** in private subnet (no direct internet)



Connect to Private EC2 via Bastion

1. Connect to Bastion Host:

[ssh -i bastion-key.pem ec2-user@ 43.205.253.242]

[ssh ec2-user@ 192.168.143.11]

Task 5: Create a NACL rule for your VPC and subnets and test the connectivity along with Security groups. "List out the differences between NACL & Security Group."

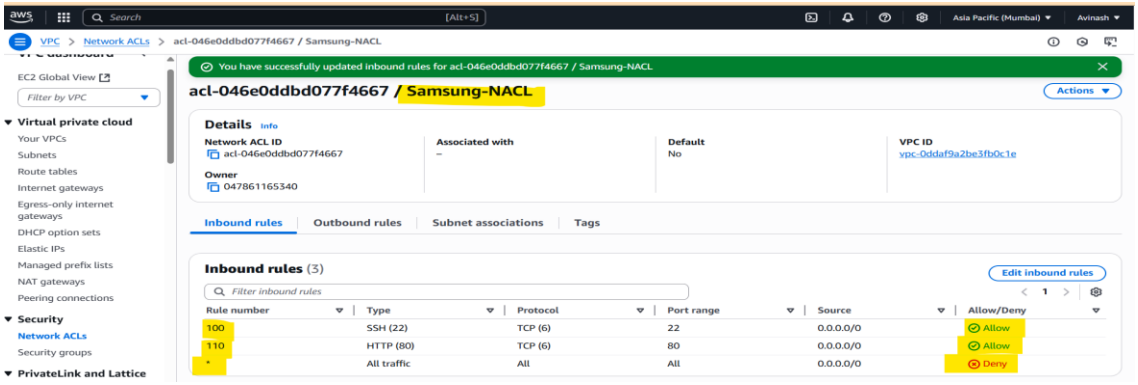
Feature	Network ACL (NACL)	Security Group (SG)
Applies To	Entire subnets	Individual instances (ENIs)
Stateless or Stateful	Stateless — return traffic must be allowed	Stateful — return traffic is auto allowed
Default Behaviour	Denies all unless rules added	Denies all unless rules added
Rule Processing	Rules are evaluated in number order	All rules are evaluated together
Allow/Deny support	Can allow or deny traffic	Can only allow traffic
Use Case	Fine control at subnet level (e.g., blacklists)	Granular access at instance level
Typical Use	Block suspicious IPs, subnet-level rules	Application-level access control

- Create NACL & Apply to Subnets

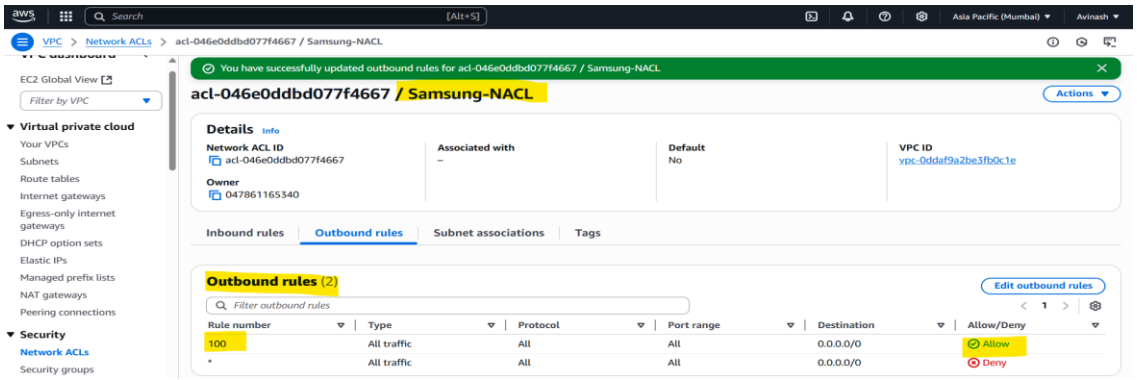
  Step 1: Go to VPC Console - AWS Console → **VPC** → **Network ACLs**

  Step 2: Create a NACL - Click **Create network ACL**- **Name tag**: Samsung-NACL-**VPC**: Select your `Samsung-VPC`- Click **Create**

Step 3: Add Inbound Rules- Click your NACL → **Inbound Rules → Edit Inbound Rules**



This allows SSH and HTTP in, and all traffic out.



Step 5: Associate NACL with Subnets

1. Go to **NACL > Subnet associations**
2. Click **Edit subnet associations**
3. Select the subnets (e.g., Samsung-Pub-1, Samsung-Priv-1)
4. Save

Step 6: Test the Connectivity

Task 6: Create a peering connection between two different VPCs

Use case a: VPC-PROD is in Mumbai Region and VPC-UAT is in Northern Virginia

Use case b: VPC-UAT & VPC-DEV both are in Mumbai Region.

Use case c: Nokia vpc is in Account A Samsung VPC is in Account B.

---

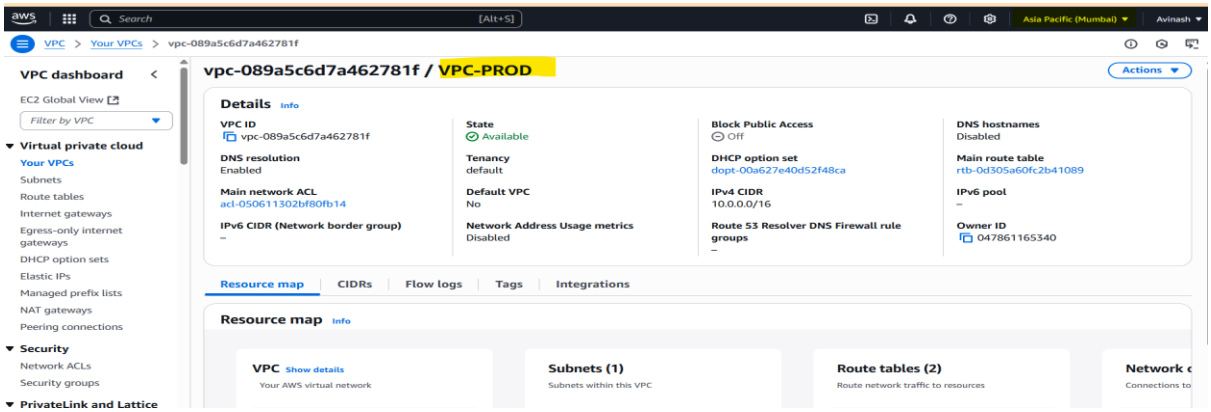
1. Create Peering from Mumbai side

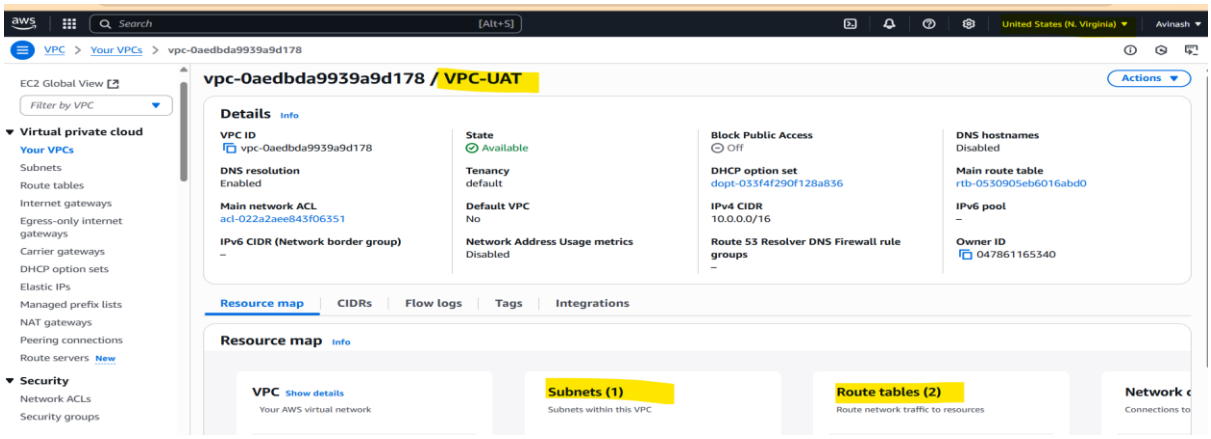
- Go to VPC Dashboard → Peering Connections → Create Peering Connection->Name tag: prod-uat-peering

Use Case	VPCs	Region	Accounts	Peering Type
A	PROD ↔ UAT	Mumbai ↔ N.Virginia	Same	Inter-region
B	UAT ↔ DEV	Mumbai	Same	Intra-region
C	NOKIA ↔ SAMSUNG	Any	Different	Cross-account

Use Case A: Inter-Region Peering :-

VPC-PROD (Mumbai - ap-south-1) ↔ VPC-UAT (N. Virginia - us-east-1)





Task 7: Create an IAM user [Adam(administrator) , Bob(network admin) , Carl(developer), Joseph(Monitoring , Tom(administrator)] and add them to the respective groups.

Assign MFA for each user and assign MFA for the root user

Create an alias for your aws account

Create an IAM role for AWS ec2 service to push log files to s3 bucket

Create an IAM role for cross account access [Auditing purpose]

User	Group	Permission Policy
Adam	Admins	AdministratorAccess
Bob	NetworkAdmins	AmazonVPCFullAccess
Carl	Developers	AmazonEC2FullAccess, S3ReadOnly
Joseph	Monitors	CloudWatchReadOnlyAccess
Tom	Admins	AdministratorAccess

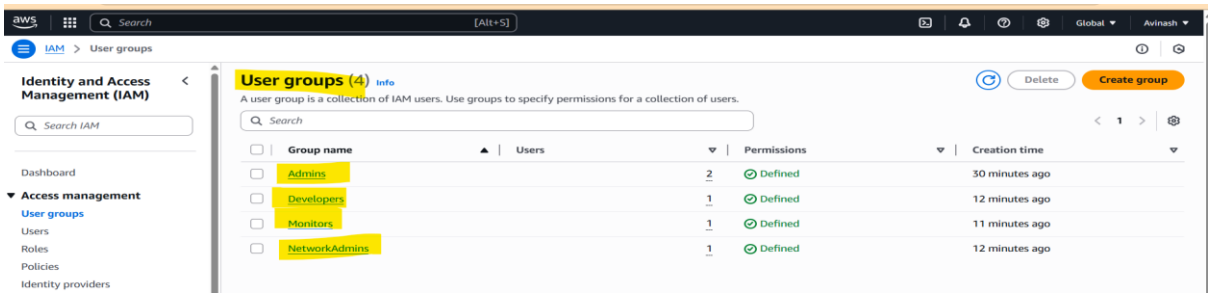
1.1 Create IAM Groups

Go to: **IAM > User groups > Create group**

Repeat for:

- Admins, NetworkAdmins ,Developers, Monitors

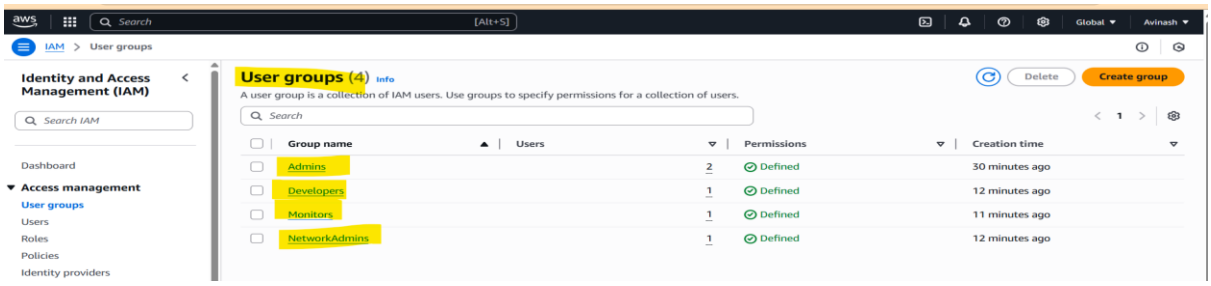
Attach appropriate policies to each group:ex-[Admins → Attach AdministratorAccess]



1.2 Create IAM Users and Add to Groups

Go to: **IAM > Users > Add users** For each user:

- **Username**: Adam, Bob, etc.
- **Access type**: Select **AWS Management Console access**
- **Password**: Auto-generate or custom
- Add to the respective **IAM group** -> Click **Create user**



2.2 Enable MFA for Root User

1. Sign in as **Root user**,Go to **IAM > Dashboard**,Under **Security recommendations**, click **Enable MFA for root**,Follow the same steps as above (virtual MFA is fine)

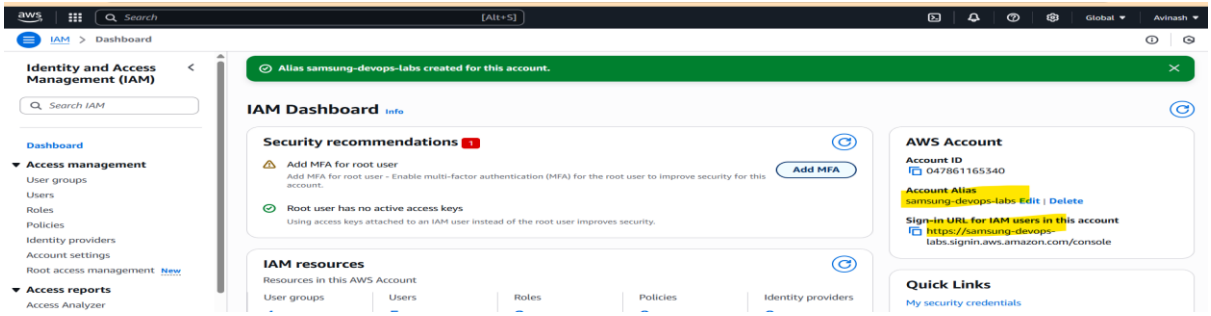
   Done. Now, even the **root account is protected by MFA**

Step 3: Set an AWS Account Alias

This makes your login URL user-friendly:

1. Go to: **IAM > Dashboard**,Under **Account Alias** → Click **Create**,Enter something like: samsung-devops-lab

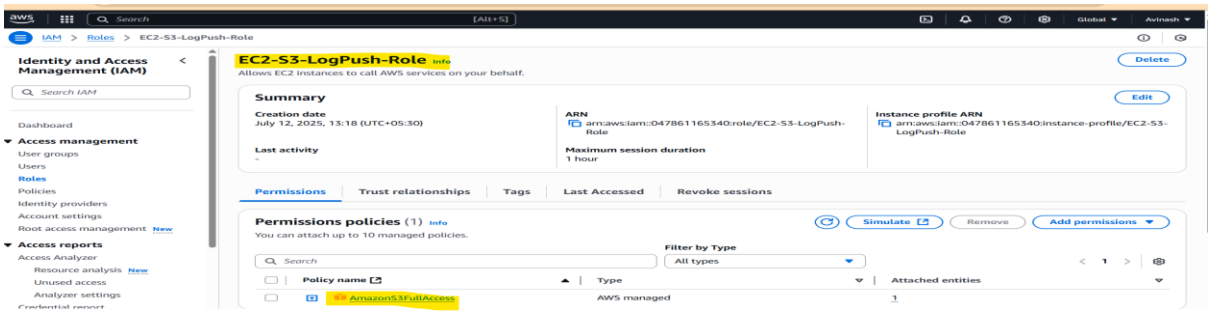
2. Click **Save**



Step 4: IAM Role for EC2 to Push Logs to S3

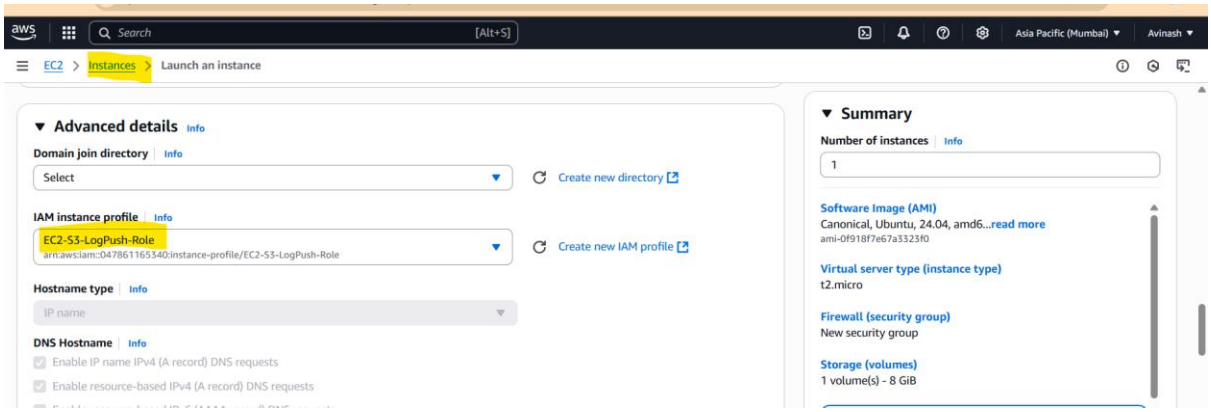
- **4.1 Create an IAM Role**

1. Go to **IAM > Roles > Create Role**, Select **Trusted Entity**: AWS service → **EC2** ,Attach Policy: AmazonS3FullAccess (or custom limited to one bucket),Name: EC2-S3-LogPush-Role -Create Role



4.2 Attach Role to EC2 Instance

1. Go to **EC2 > Instances > [Your EC2 instance]**,Click **Actions > Security > Modify IAM role**,Select EC2-S3-LogPush-Role-Save



Task 8:Create an Ec2 instance with ami ubuntu/amazon linux. Then install apache webserver with userdata: userdata Documentation. Then take backup of the ec2 using below options.

Option a: Create Image [Manual Effort]

Option b: Create Snapshots from EBS volume and then create volume and then attach it to the instance. [Manual Effort]

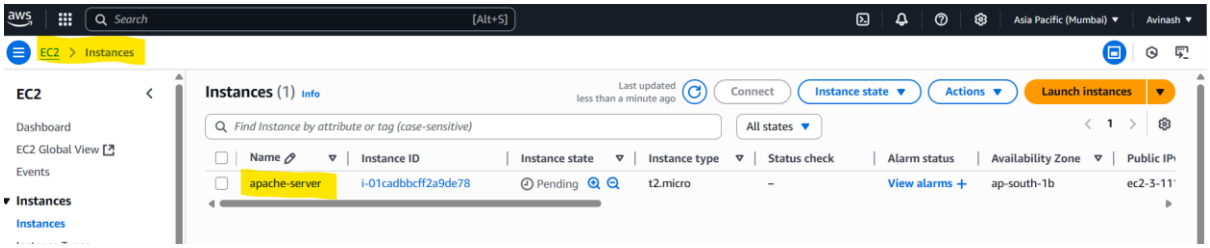
Option c: Use aws backup service to automate the backup [Automation]

Step 1: Launch EC2 Instance

1. Go to: **EC2 > Launch Instance**
2. Fill:**Name**: Apache-Server **AMI**: Choose either:Amazon Linux 2 -Ubuntu 20.04 LTS (your choice)
    - **Instance type**: t2.micro (Free tier)
    - **Key pair**: Select or create one
3. **Network settings**:VPC: Choose your existing VPC,Subnet: Select public subnet

    Auto-assign public IP: **Enable**

    - Security Group:
        - Allow **SSH (22)** from your IP
        - Allow **HTTP (80)** from anywhere (0.0.0.0/0)



Step 2: Add Apache Web Server via User Data

Click **Advanced details > User data**
Paste the appropriate script:

#!/bin/bash

apt update -y

apt install apache2 -y

systemctl start apache2

systemctl enable apache2

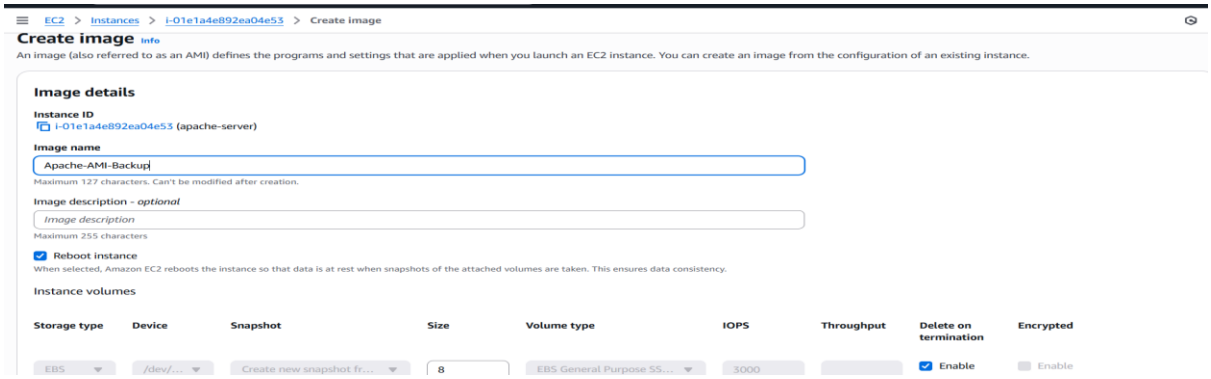
echo "Welcome to Apache on Ubuntu" > /var/www/html/index.html

Part 2: Backup Options

Option A: Create Image (Manual AMI Backup)

1. Go to EC2 > Instances -Select your instance-Click Actions > Image and templates > Create image
2. Fill:Name: Apache-AMI-Backup,No reboot: leave unchecked (optional)
3. Click Create image Wait 2–3 minutes for the AMI to complete

You can later launch new instances from this AMI**.**

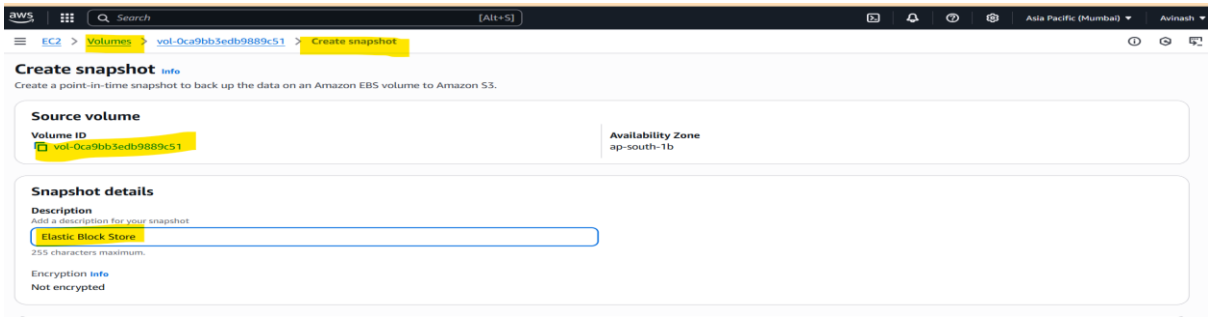


Option B: EBS Snapshot and Restore (Manual)

Step 1: Take Snapshot of Root Volume

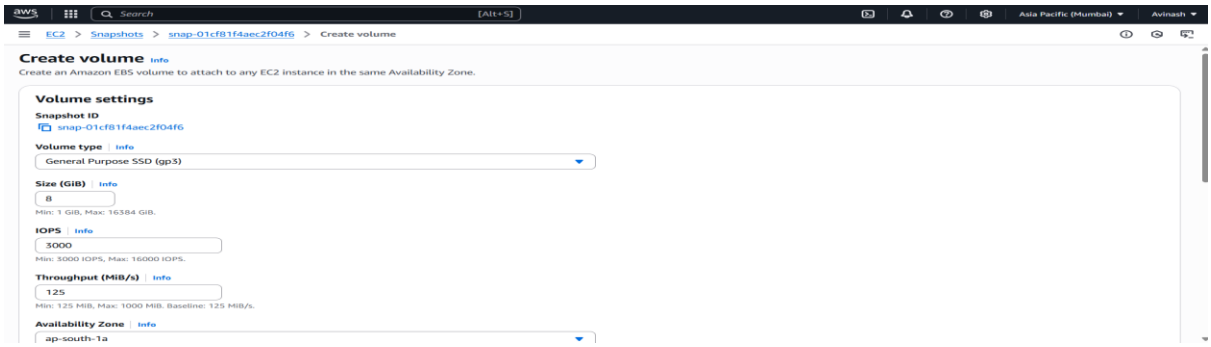
1. Go to: **EC2 > Instances**-Select your instance-In **Storage** tab, copy the **Volume ID**

2. Go to: **Elastic Block Store > Volumes** -Select the Volume → Click **Actions > Create Snapshot**-Name: Apache-Volume-Snapshot



Step 2: Create Volume from Snapshot

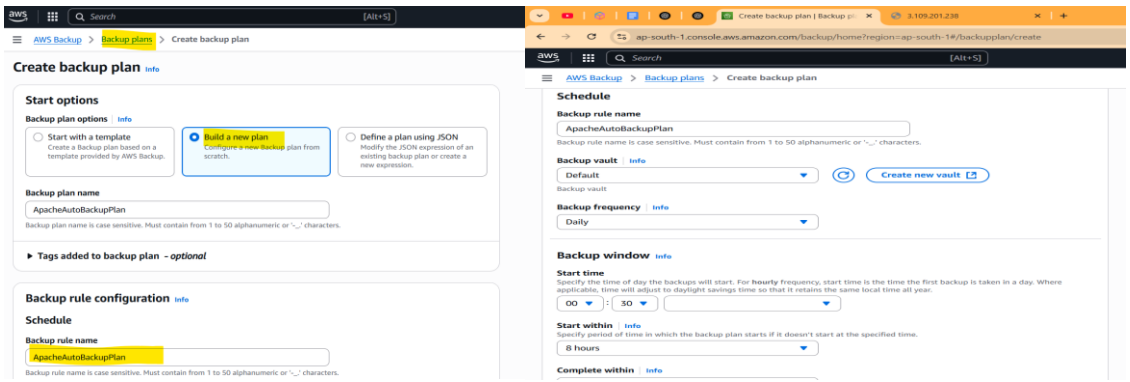
1. Go to: **Snapshots**-Select your snapshot → Click **Actions > Create Volume**
2. Choose:Availability Zone: must match EC2's AZ (e.g., us-east-1a)
3. Click **Create Volume**



Option C: Use AWS Backup (Automation)

Step 1: Configure AWS Backup

1. Go to: **AWS Backup > Backup plans**-Click **Create backup plan**-Choose: **Build a new plan**-Name: ApacheAutoBackupPlan
2. Add rule:Rule name: DailyBackup-Frequency: Daily-Retention: 7 days-Backup window: Default



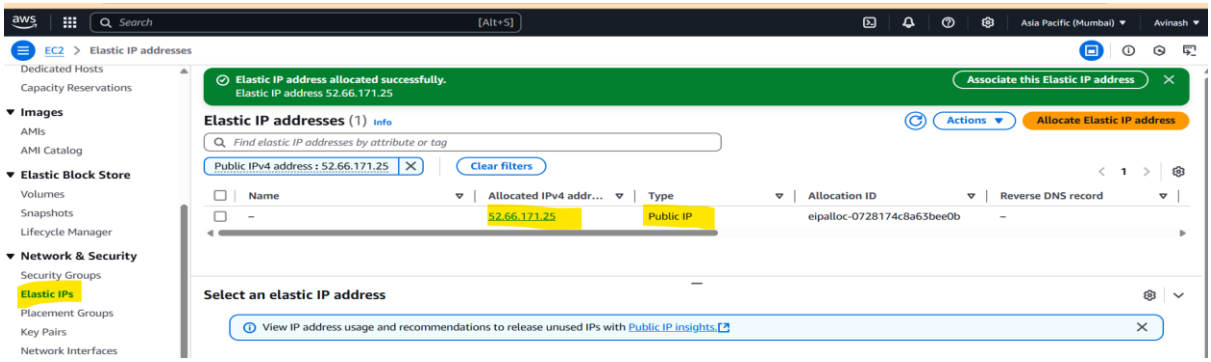
Task 9: Associate EIP for the instance and test whether the ip is constant even if the instance is stopped/terminated. Release back the EIP back to aws

---

Step 1: Allocate an Elastic IP

1. Go to **EC2 Console**-On the left sidebar, click **Elastic IPs**
2. Click **Allocate Elastic IP address**
3. Leave default settings:-**Scope**: Amazon's pool of IPv4 addresses -Click **Allocate**

   A new Elastic IP will be created (e.g., 43.204.x.x)

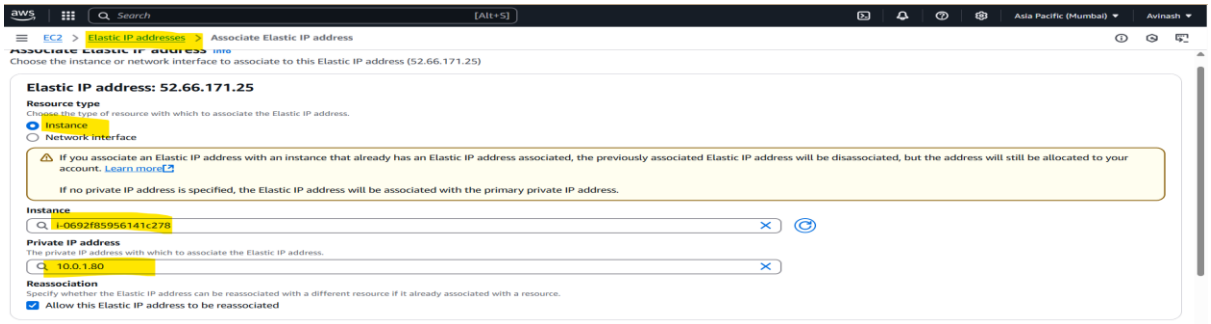


Step 2: Associate the EIP with Your EC2 Instance

1. Select the newly allocated Elastic IP-Click **Actions > Associate Elastic IP address .**
2. Choose:**Resource type**: Instance-**Instance**: Select your running EC2 instance

   Private IP: Leave default-Click **Associate**

Now your instance has a **static public IP** (EIP).



Step 3: Test Elastic IP Behavior - Test 1: Stop and Start Instance

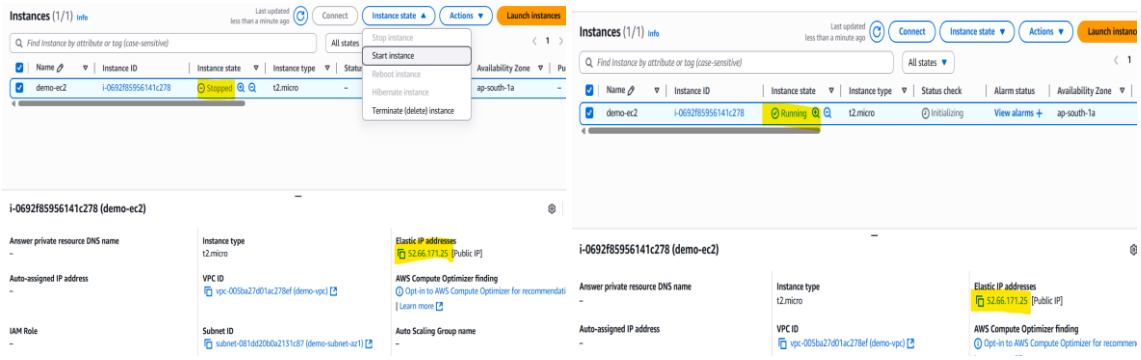
1. Go to **EC2 > Instances** - Stop your instance-Wait until status = stopped

   Start the instance again-**Elastic IP should stay the same.**

2. **Test 2: Terminate Instance**

1. Select the instance → Click **Actions > Instance State > Terminate**
2. Wait until it's fully terminated -**Elastic IP becomes unassociated**

- It's still in your account, but no longer linked to any instance



Task 10: Launch an EC2 instance in Mumbai region create additional EBS volume of 5GB and attach to the instance and check whether the disk is added properly or not

---

Make sure you are in the **Mumbai region (ap-south-1)**

Step 1: Launch EC2 Instance-Go to **EC2 Console-**Click **Launch Instance**

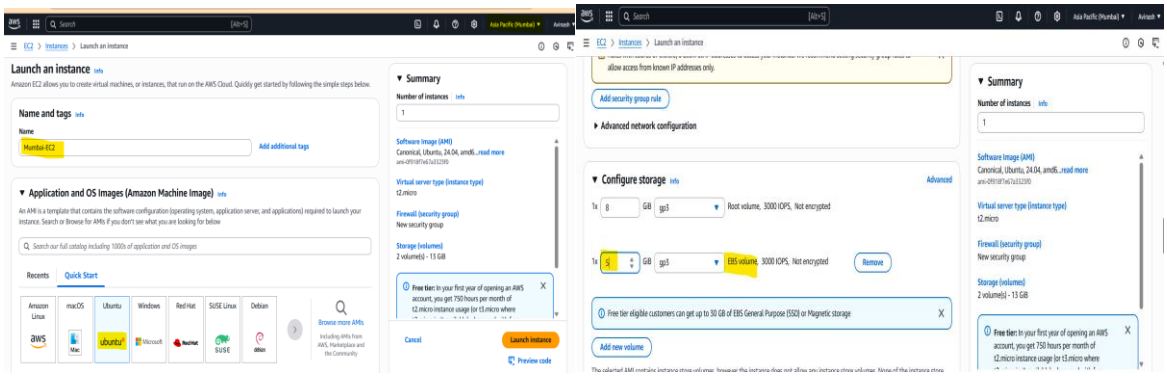
1. Fill:**Name**: Mumbai-EC2-**AMI**: Amazon Linux 2 (or Ubuntu)-**Instance Type**: t2.micro (free tier)-**Key Pair**: Select or create new
2. Under **Network Settings**:Choose your VPC & subnet (e.g., public subnet)
    - Enable **Auto-assign Public IP**
    - Allow **SSH (port 22)** from your IP
3. Click **Launch Instance**

Step 2: Create Additional EBS Volume (5 GB) -Go to **EC2 > Elastic Block Store > Volumes-**Click **Create Volume**

1. Fill:**Size**: 5 GiB-**Availability Zone**: Match **your EC2 instance's AZ**

    Example: If EC2 is in ap-south-1a, volume must be in ap-south-1a

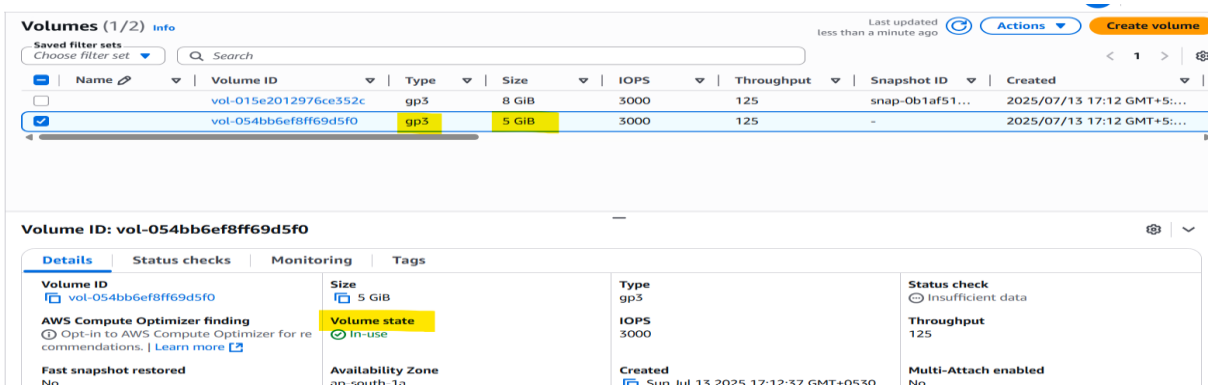
    - **Volume Type**: gp3 (recommended)
    - Optional tag: Name: ExtraVolume
2. Click **Create Volume**



Step 3: Attach Volume to the EC2 Instance

1. Go to: **Volumes > Select your 5 GB volume**-Click **Actions > Attach Volume**
2. Fill:**Instance**: Select your Mumbai EC2 instance-**Device name**:

   Use: /dev/xvdf (for Amazon Linux) or /dev/sdf (Ubuntu)
3. Click **Attach**



Step 4: Connect and Verify Disk Inside EC2

CMD: ssh -i "avi-key.pem" ubuntu@ 35.154.237.112

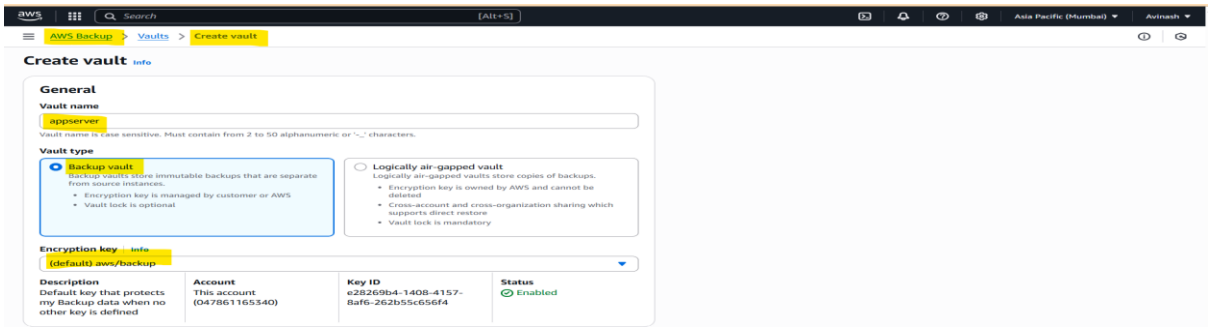
CMD: lsblk

```
ubuntu@ip-10-0-1-242:~$ lsblk
NAME     MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0      7:0    0 27.2M  1 loop /snap/amazon-ssm-agent/11320
loop1      7:1    0 73.9M  1 loop /snap/core22/1981
loop2      7:2    0 50.9M  1 loop /snap/snapd/24505
xvda     202:0    0    8G  0 disk
├─xvda1  202:1    0    7G  0 part /
├─xvda14 202:14   0    4M  0 part
├─xvda15 202:15   0  106M  0 part /boot/efi
└─xvda16 259:0    0  913M  0 part /boot
xvdb     202:16   0    5G  0 disk
ubuntu@ip-10-0-1-242:~$
```

Task 11: Create an AWS BACKUP plan to take backup of your company's application server to run at 11:30 PM everyday and store the backup at the vault name "appserver"

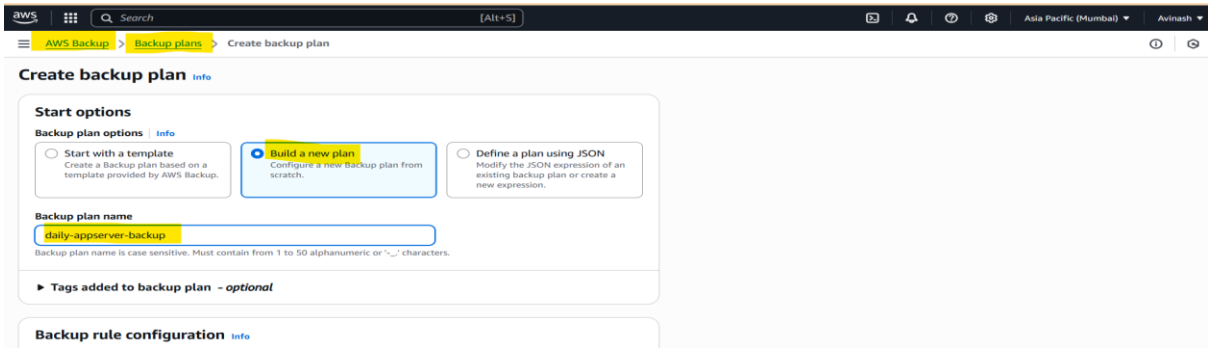
Step 1: Create a Backup Vault

1. Go to the **AWS Backup Console** [https://console.aws.amazon.com/backup]
2. On the left menu, click **Backup vaults**-Click **Create backup vault**
3. Enter **Name**: appServer- Click **Create backup vault**



Step 2: Create a Backup Plan

1. Go to **Backup plans** in the left menu- Click **Create backup plan**-Choose **Build a new plan .**
2. Enter:**Backup plan name**: daily-appserver-backup

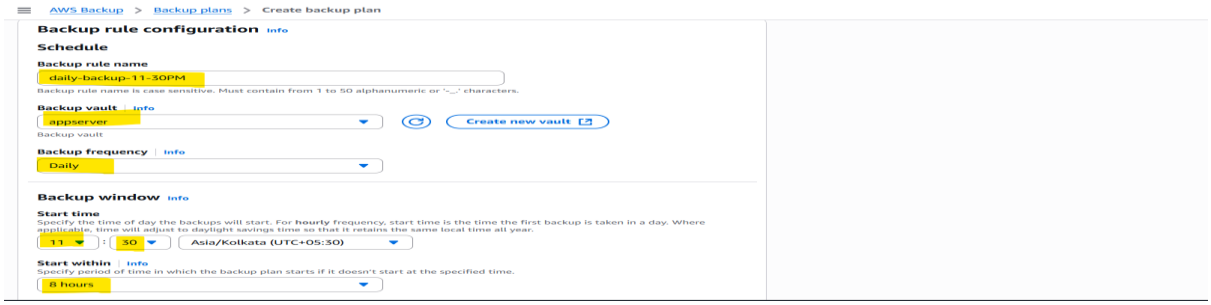


Step 3: Configure Backup Rule

In **Backup rule configuration**:**Rule name**: daily-backup-11-30PM,**Backup frequency**: Daily **Backup window**:

- Start time: 11:30 PM
- Retention period: Set how long you want to keep backups (e.g., 7 days, 30 days)

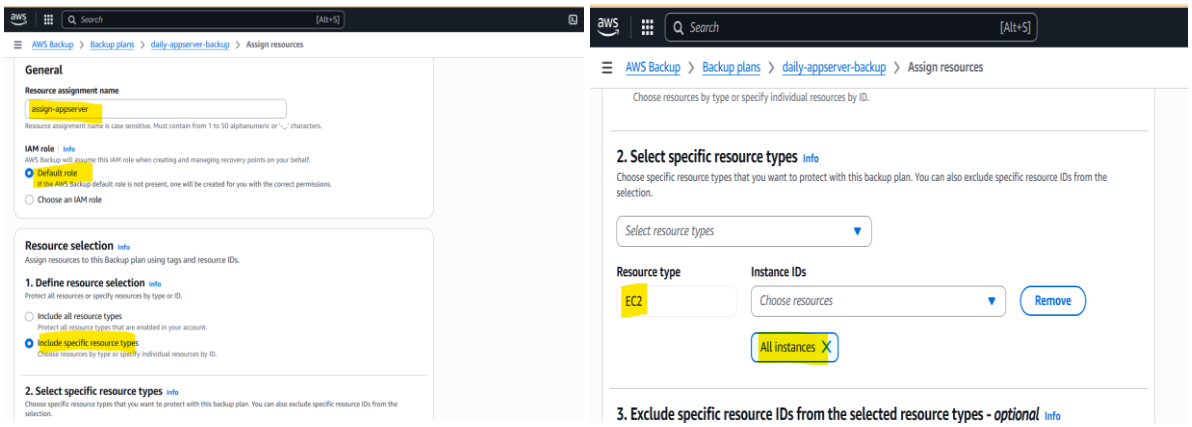
2. **Backup vault**: Select appserver- Click **Add backup rule** - Click **Create plan**



Step 4: Assign Resources (Tag or ARN Based)

After creating the plan:

1. Go to **Backup plans → daily-appserver-backup**- Click **Assign resources**
2. Enter:**Resource assignment name**: assign-appserver
3. Choose:**IAM role**: Use default or create a new one,**Assign by**: Use **Resource ID** (select EC2 instance directly)
   OR use **Tags** like: Key: Backup, Value: Daily
4. Select your **EC2 application server** instance- Click **Assign**



Step 5: Monitor Backups

- Go to **Backup Jobs** to monitor backup executions
- Backup will run every day at **11:30 PM** and store in the **appserver vault**

Task 12: Create an Application load balancer to demonstrate path based routing.

Example: http://apache-webserver-asg-1-1138403636.ap-south-1.elb.amazonaws.com/ → Main Page

http://apache-webserver-asg-1-1138403636.ap-south-1.elb.amazonaws.com/console/ → redirect to aws console

http://apache-webserver-asg-1-1138403636.ap-south-1.elb.amazonaws.com/gfg/ → redirect to gfg

---

I want to set up:

- A Load Balancer that routes based on **path**
- Example paths:
    - / → Main page
    - /console/ → Redirect to AWS Console
    - /gfg/ → Redirect to GeeksforGeekss

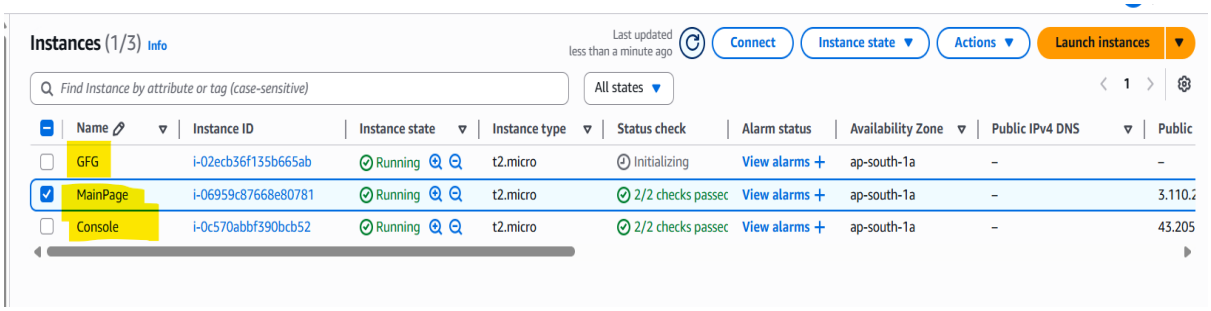
AWS Services Involved

- **EC2 instances** (running Apache or NGINX)
- **Launch Templates or Auto Scaling Group** (optional but ideal)
- **Application Load Balancer (ALB)**
- **Target Groups** (for path-based routing)
- **Listener Rules** (to route paths)

Step 1: Launch 3 EC2 Instances

Each instance will serve different content:

Instance Name	Purpose	Example Content
MainPage	/	Main page (index.html)
Console	/console/	Redirects to AWS Console
GFG	/gfg/	Redirects to GeeksforGeeks



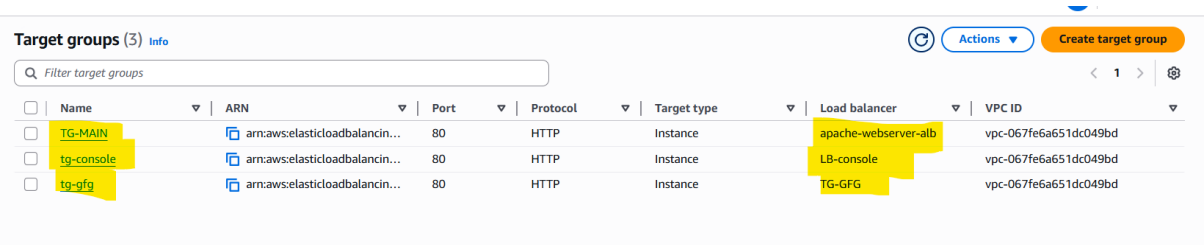
5. Modify /var/www/html/index.html with unique content on each EC2:

- Instance 1 → Main Page
- Instance 2 → <meta http-equiv="refresh" content="0; url=https://console.aws.amazon.com">

Create 3 Target Groups

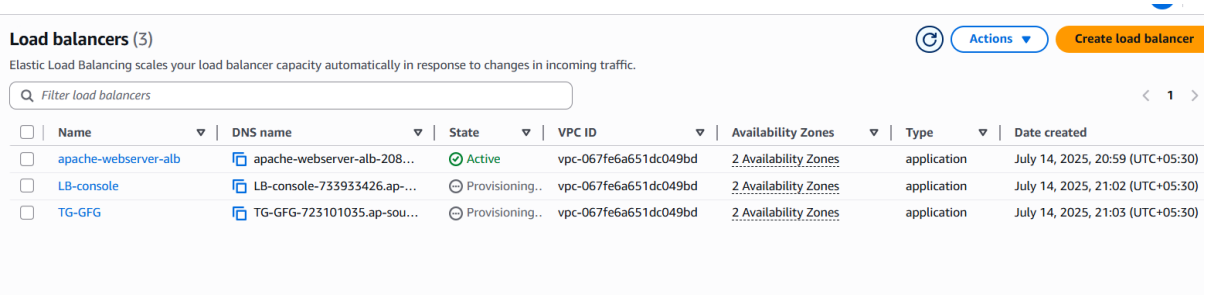
Go to **EC2 → Target Groups** → Click **Create target group**

Name	Protocol	Target Type	Attach
tg-main	HTTP	instance	MainPage EC2 instance
tg-console	HTTP	instance	Console EC2 instance
tg-gfg	HTTP	instance	GFG EC2 instance



Step 3: Create Application Load Balancer (ALB)

1. Go to **EC2 → Load Balancers → Create Load Balancer**-Choose **Application Load Balancer**-Enter name like: apache-webserver-alb
2. Scheme: **Internet-facing**-IP type: **IPv4**-Listeners: **HTTP port 80**
3. Availability Zones: Select VPC + at least 2 subnets-Click **Next**



Step 5: Add Listener Rules (Path-Based Routing)

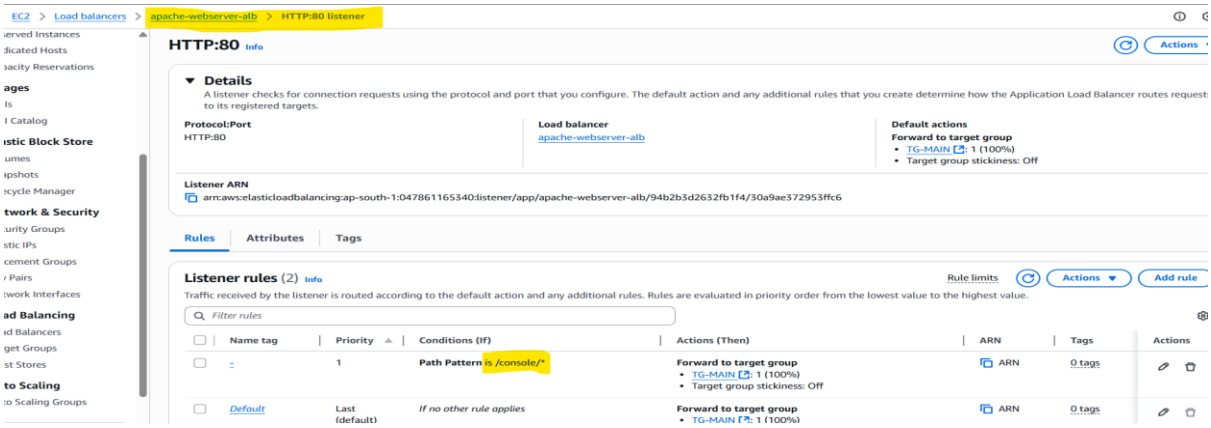
After ALB is created:

1. Go to **EC2 → Load Balancers → Select your ALB**

2. Click on **Listeners → View/Edit rules** (on port 80)- Add rules like this:

Path Pattern	Target Group
/console/*	tg-console
/gfg/*	tg-gfg
/ (Default rule)	tg-main

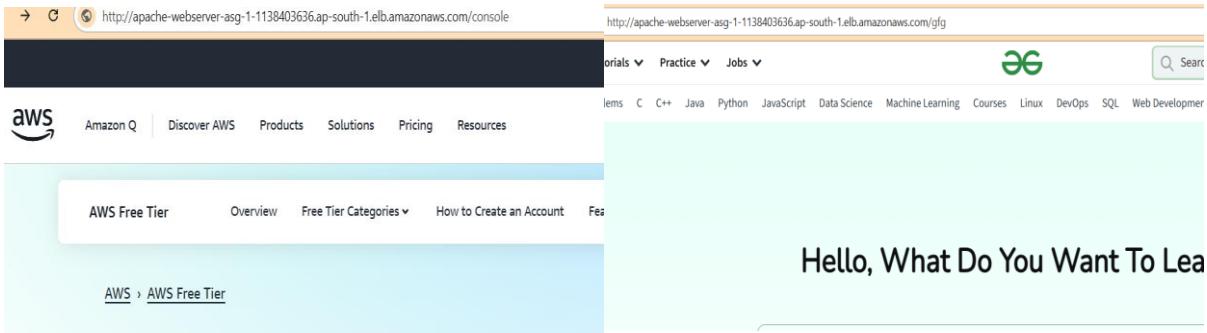
Make sure rule priority is set accordingly (lower number = higher priority).



Step 6: Test the URLs

Use the **DNS name** of the ALB:

http://apache-webserver-asg-1-1138403636.ap-south-1.elb.amazonaws.com/

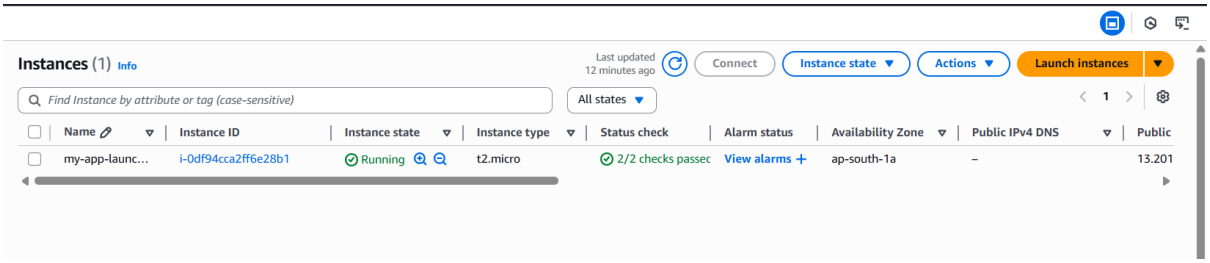


URL	Output
/	Shows your custom Main page
/console/	Redirects to AWS Console
/gfg/	Redirects to GeeksforGeeks

Task 13: Create an Autoscaling group for your existing application load balancer which should auto scale based on CPU utilization.

Step 1: Create a Launch Template

1. Go to **EC2 Console → Launch Templates**
2. Click **Create launch template**

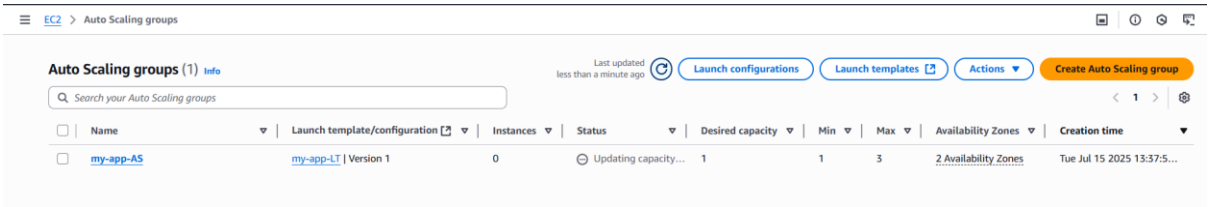


Step 2: Create Auto Scaling Group (ASG)

1. Go to **EC2 → Auto Scaling Groups**
2. Click **Create Auto Scaling group**

Fill in the details:**Name**: my-app-asg -**Launch Template**: Select the one you created above

3. **Network**:Select your **VPC**-Choose **2 or more subnets** (same ones used by your ALB)



Step 5: Configure Scaling Policy (Based on CPU)

Choose:

- **Target tracking scaling policy**
- Metric: Average CPU utilization
- Target value: 50 (or any % based on your threshold)

This means:

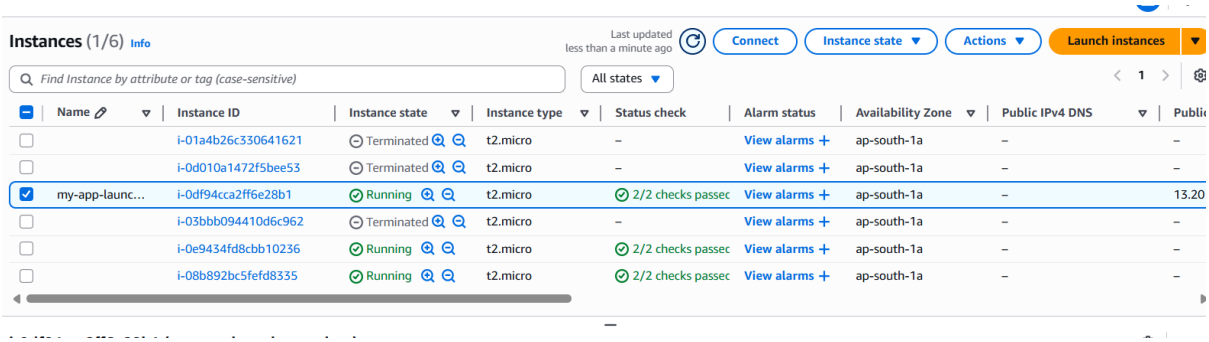
If average CPU > 50% → Scale out
If average CPU < 50% → Scale in

Testing the Auto Scaling

- Use a **stress tool** (like stress or CPU spike script) to simulate CPU load:

sudo yum install stress -y

stress --cpu 2 --timeout 120



- Watch **EC2 → Auto Scaling Group → Activity history**
- It should launch a new instance if CPU > threshold

Task 14: Create a Network Load balancer to distribute incoming traffic to traccar software.

Linux Installation - Traccar Follow this instruction to install traccar software

Jenkins installation follow the userdata script here →

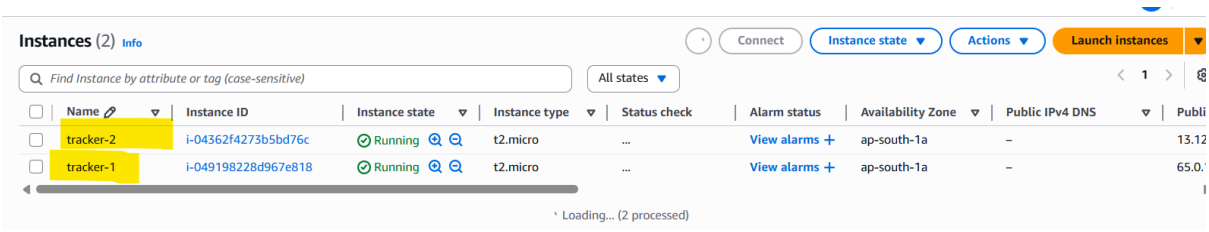
Step 1: Launch EC2 Instances

You need **2 or more Linux EC2 instances** for high availability behind the NLB.

Launch EC2 instances with:

- OS: **Ubuntu 20.04 or Amazon Linux 2** ,Ports Open: **22**, **8082** (Traccar Web UI)
- Add a **Tag**: Key=App, Value=traccar (optional but helpful for filtering)
- Assign **Public IP** only for SSH if testing outside

After launch, SSH into each instance and install Traccar manually (next step)



Step 2: Install Traccar Software (on each EC2)

Traccar Installation Steps (Linux):

wget https://github.com/traccar/traccar/releases/download/v6.8.1/traccar-linux-64-6.8.1.zip

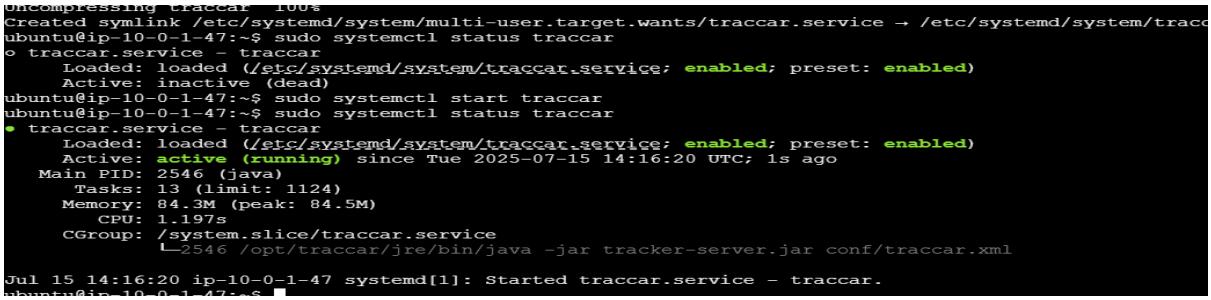
sudo apt install unzip -y

unzip traccar-linux-64-6.8.1.zip

sudo ./traccar.run

Confirm service is running:[**sudo systemctl status traccar]**

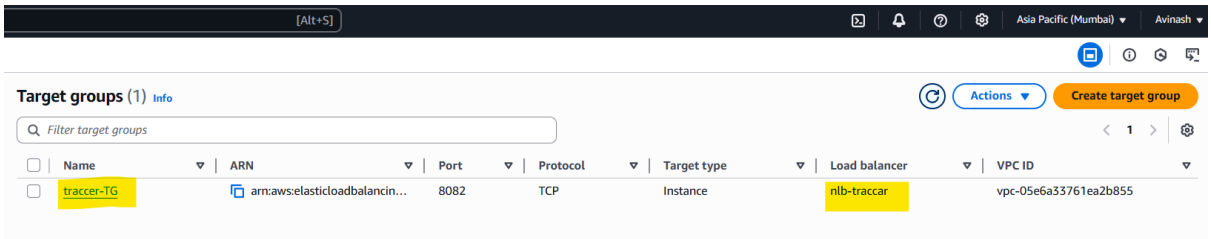
[sudo systemctl start traccar]



Step 3: Create Target Group (TCP)

1. Go to **EC2 → Target Groups → Create Target Group**-Type: **Instances**
2. Protocol: **TCP**,Port: **8082**,VPC: Select the one used by your EC2s
3. Name: tg-traccar -Click **Next**, then:

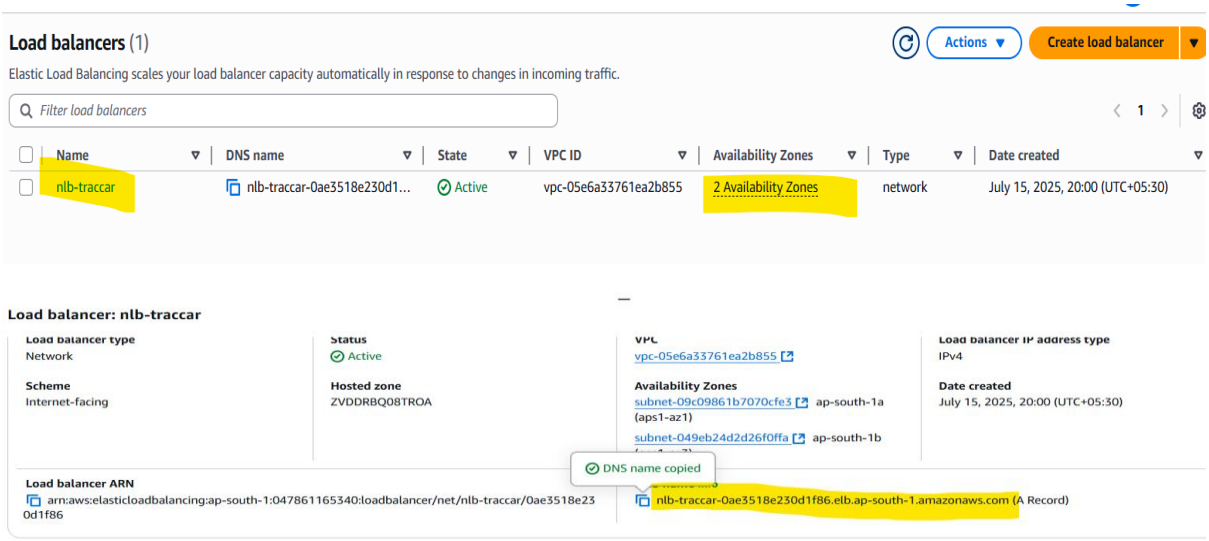
- Select the EC2 instances running Traccar,Click **Register**,Click **Create target group**



Step 4: Create Network Load Balancer (NLB)

1. Go to **EC2 → Load Balancers → Create Load Balancer.**
2. Choose **Network Load Balancer**,Name: nlb-traccar,Scheme: Internet-facing,IP type: IPv4.
3. Listeners:Protocol: **TCP** ,Port: **8082**,Availability Zones: Select at least 2 subnets (same as EC2 placement).

4. Forwarding to Target Group: Choose tg-traccar Click **Create Load Balancer**

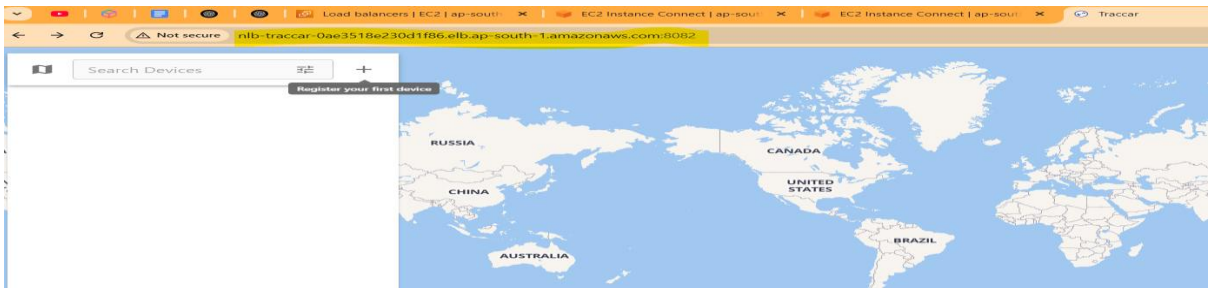


Step 5: Test the Load Balancer

- Copy the **DNS name** of the NLB (e.g. nlb-traccar-123456.elb.amazonaws.com)
- Open in browser:

http://nlb-traccar-0ae3518e230d1f86.elb.ap-south-1.amazonaws.com:8082

You should see the **Traccar login page**



Must we need to allow ufw: sudo ufw allow 8082, sudo ufw enable

Install Jenkins by using script
#!/bin/bash

sudo apt update -y

sudo apt install openjdk-11-jdk -y

wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -

sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ > /etc/apt/sources.list.d/jenkins.list'

sudo apt update -y

sudo apt install jenkins -y

sudo systemctl enable jenkins

sudo systemctl start Jenkins

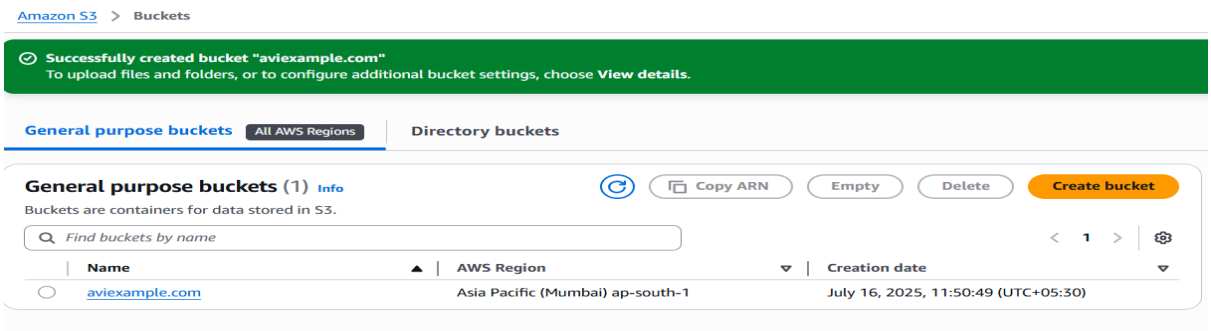
Task 15: Host a static website for your company domain (eg: example.com) using S3, Route 53.

1. Create a lifecycle rule for the S3 bucket to transition the objects to S3 Glacier deep archive after 60 days and after transition to S3 Glacier deep archive the objects should be deleted after 6 months.
2. Secure the website by creating a cloudfront distribution and associating with AWS certificate manager.
3. Integrate ACM with ALB and route traffic to EC2 instances.

PART 1: Host Static Website on S3

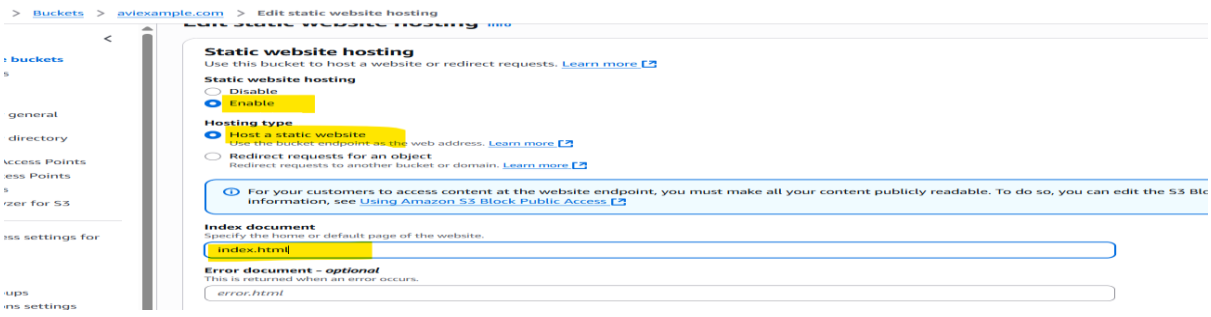
Step 1: Create S3 Bucket

1. Go to **S3 > Create bucket**,Name: example.com (must match domain).
2. Region: Select your preferred,**Uncheck** “Block all public access”,Create the bucket.



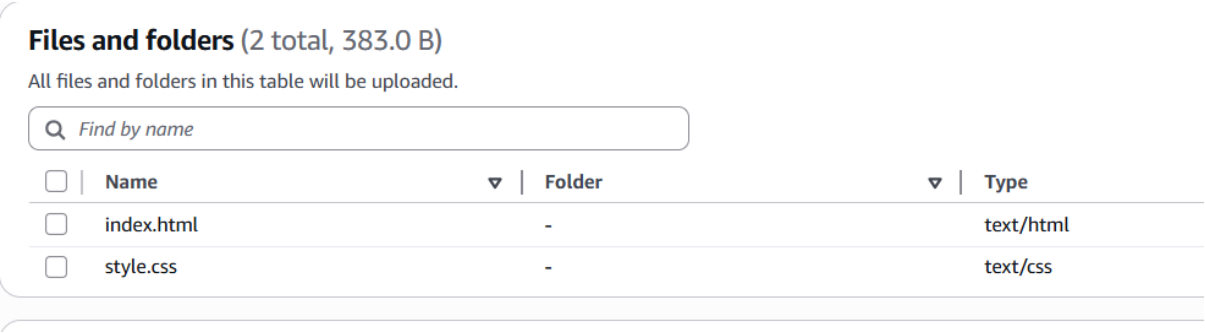
Step 2: Enable Static Website Hosting

1. Go to bucket → **Properties**,Scroll to **Static website hosting**
2. Enable it → choose:Hosting type: **Host a static website**,Index document: index.html – Save



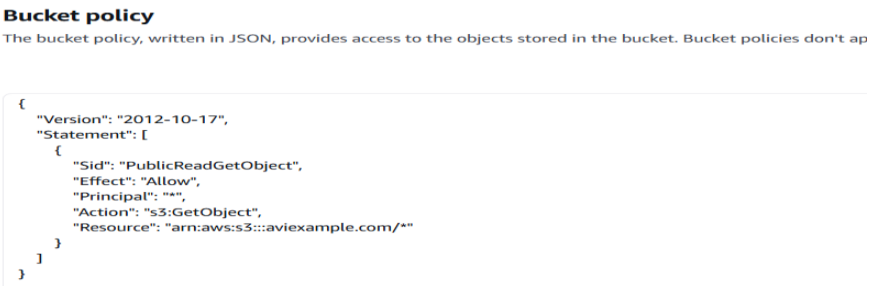
Step 3: Upload Website Files

- Upload your index.html, style.css, etc. to the S3 bucket



Step 4: Make Bucket Contents Public

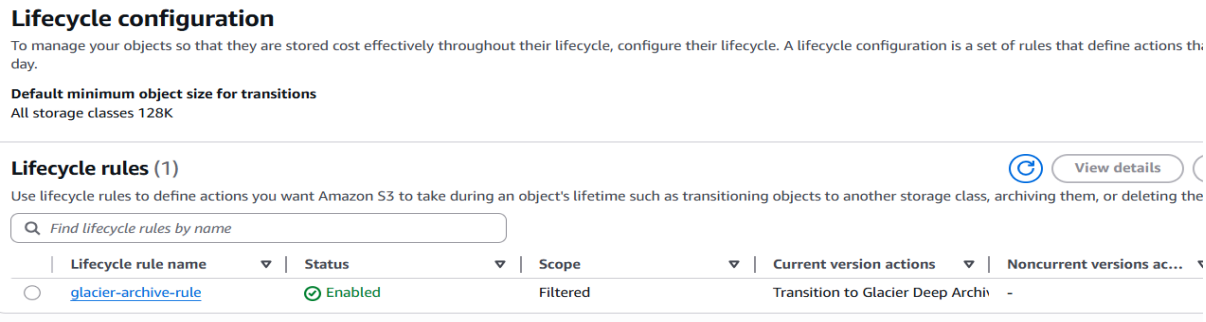
1. Go to **Permissions > Bucket policy**,Add the policy:



PART 2: Add Lifecycle Rule

Step 1: Create Lifecycle Rule

1. Go to bucket → **Management > Lifecycle rules**
2. Create rule:Rule name: glacier-archive-rule,Choose prefix: * (all objects)
    - **Transition after 60 days** → to **Glacier Deep Archive**
    - **Expire after 180 days** (6 months)



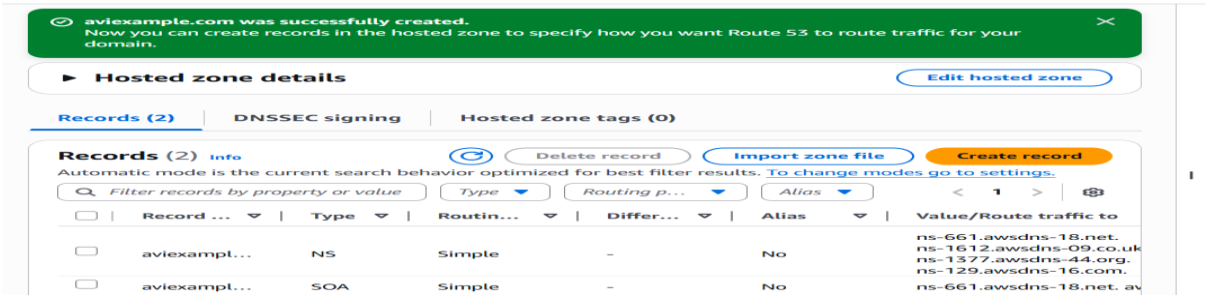
PART 3: Route 53 Domain Setup

Prerequisite: Domain

- You must own a domain (e.g., from Route 53, GoDaddy, Namecheap, etc.)

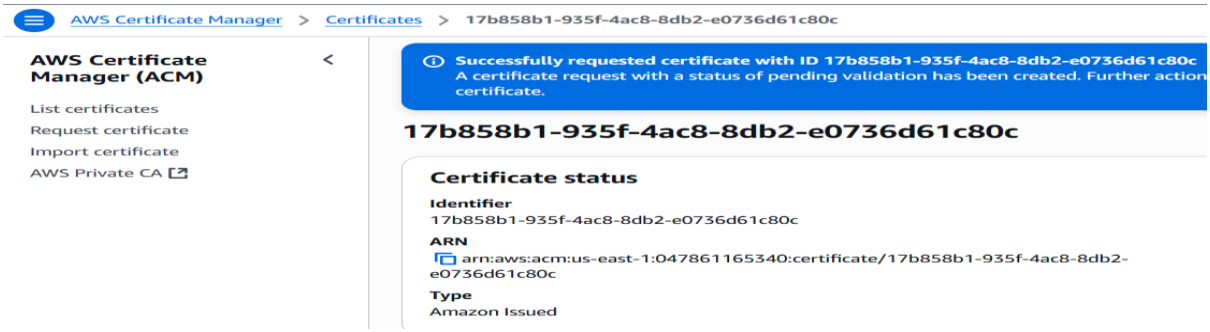
Step 1: Create Hosted Zone -Go to **Route 53 > Hosted Zones**

1. Click **Create Hosted Zone**,



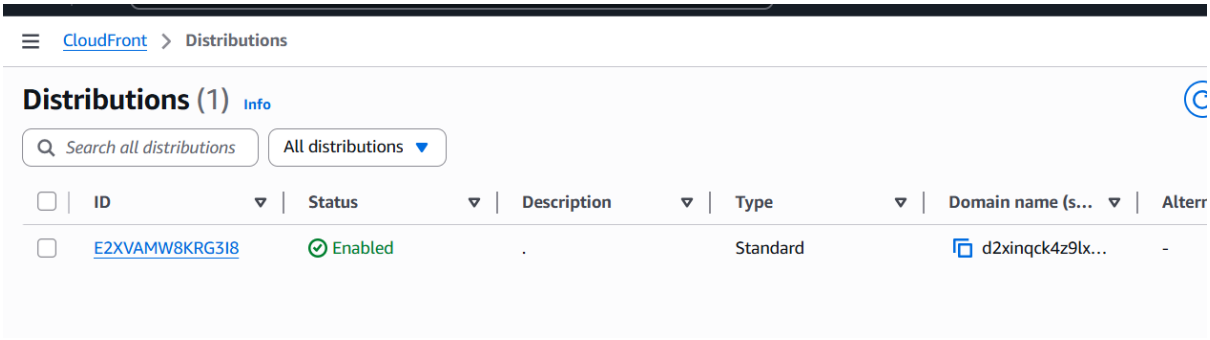
Step 2: Create Record to Point to S3

2. In the hosted zone, click **Create record** ,Type: A,Name: leave blank (for root domain),**Alias**: Yes,Target: Choose **S3 static website endpoint**
3. Domain: example.comType: Public hosted zone



Step 2: Create CloudFront Distribution

1. Go to **CloudFront > Create Distribution**
2. Origin:Origin domain: your **S3 static site endpoint** ,Origin protocol: HTTP only
3. Default behavior:Viewer protocol: Redirect HTTP to HTTPS
4. Alternate domain (CNAME): example.com
5. SSL certificate: Choose your **ACM certificate**



PART 5: ALB + ACM for EC2-based Dynamic Site

Step 1: Create ALB

1. Go to **EC2 > Load Balancers > Create Load Balancer**
2. Choose **Application Load Balancer** -Add listener on port 443 (HTTPS)
3. Add HTTPS listener and choose the **ACM certificate**
4. Select two subnets in different AZs
5. Create new target group for EC2 (HTTP or TCP)
6. Register your EC2 instances to the target group
7. Add rules to forward to target group

Step 2: Add DNS Entry for EC2 Site

1. In **Route 53**, add a new A record (e.g., aviexample.com)
    - Alias to the **ALB DNS name**

Task 16: Create an RDS Database using mysql CE and connect to the database using mysql workbench(you can download workbench here MySQL :: Download MySQL Workbench).

Conditions: The RDS has to be private and you need to connect to the DB using tcp over SSH.
[For this you need to create a BASTION HOST and pass those credentials]

Once connected, run the below SQL queries to perform CRUD operations.

Important note: Before running the queries goto EDIT → Preferences → SQL Editor and uncheck safe updates and reconnect. This is to perform DELETE operations.

show databases;

create database mtt;

use mtt;

Create table student(name Varchar(30) NOT NULL, marks Integer);

select * from student;

INSERT INTO student(name, marks)

VALUES ('Mini', 70);

DELETE FROM student WHERE (name = 'Mithun' AND marks=56);

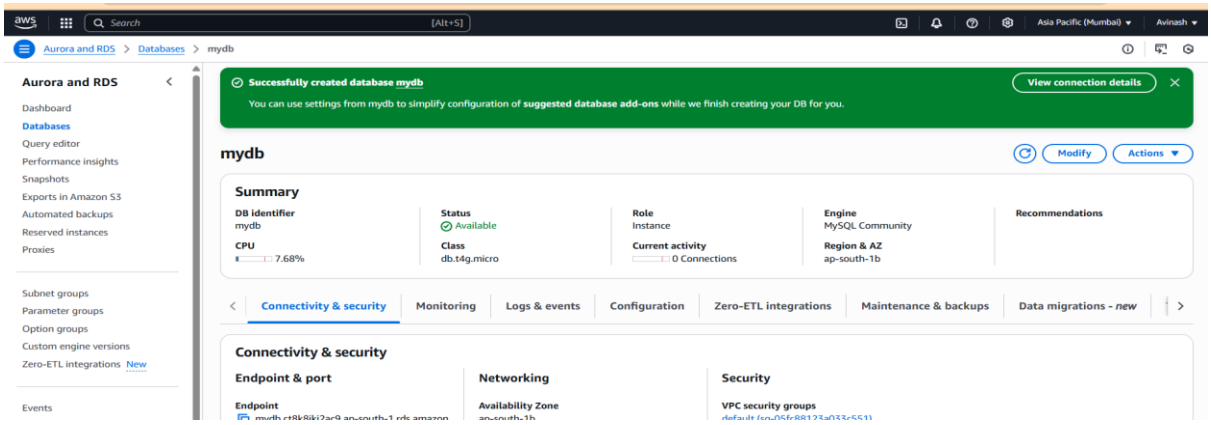
UPDATE student

SET name='Vinuth'

WHERE marks=70;

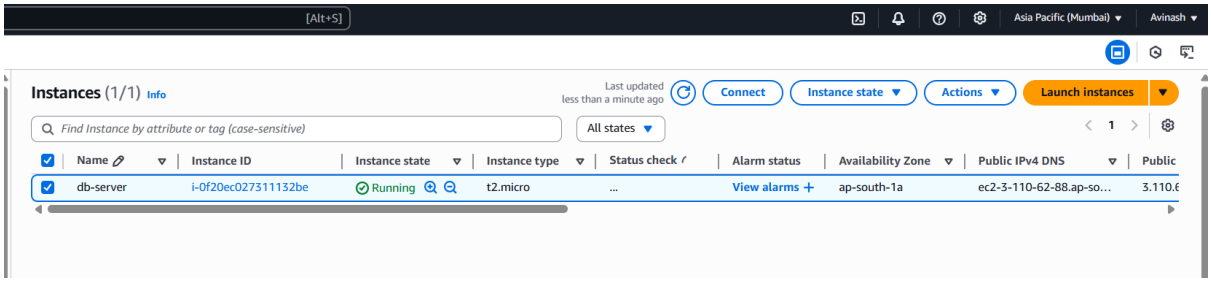
Step 1: Create a Private RDS (MySQL CE)

1. Go to **RDS > Databases > Create database**
2. Select:**Engine**: MySQL,**Edition**: Community Edition (CE),**Version**: Any recent stable
3. Settings:DB instance identifier: mydb,Master username: admin,Password: your-password
4. DB Instance Class: db.t3.micro (Free Tier if available)
5. **Storage**: General Purpose (GP2 or GP3), 20GB
6. **Connectivity**:VPC: Select your VPC,Subnet group: Choose private subnets,Public access: **No**,VPC security group: Create or choose a security group allowing port **3306** from Bastion host's SG
7. Click **Create database** (wait ~5 mins)



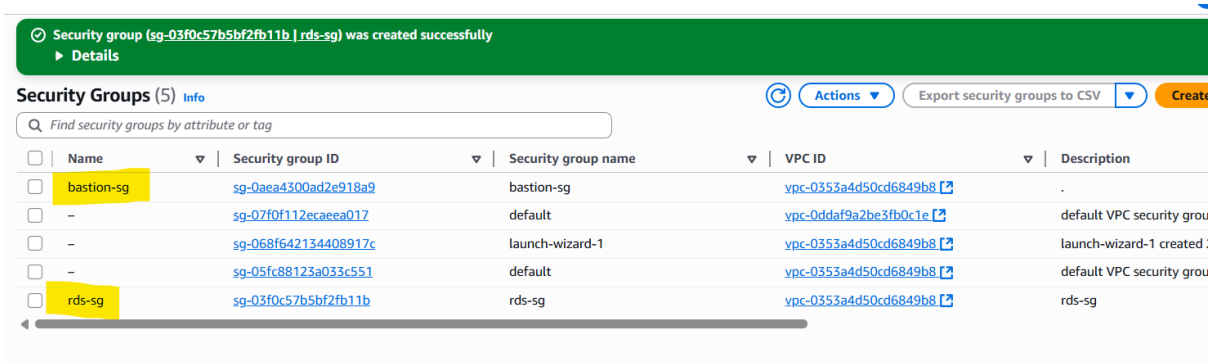
Step 2: Launch a Bastion Host (EC2 instance in public subnet)

1. Go to **EC2 > Launch Instance**,AMI: Ubuntu or Amazon Linux
2. Network settings:VPC: Same as RDS,Subnet: **Public subnet**,Auto-assign Public IP: **Enable**,Security Group:Allow **port 22 (SSH)** from your **public IP**
3. Launch instance and note:**Public IP of Bastion Host**,**Private IP of RDS instance**



Step 3: Security Group Setup

- **Bastion Host SG**:Inbound: Allow SSH (port 22) from your IP
- **RDS SG**:Inbound: Allow **port 3306 (MySQL)** only from the **Bastion Host's security group**

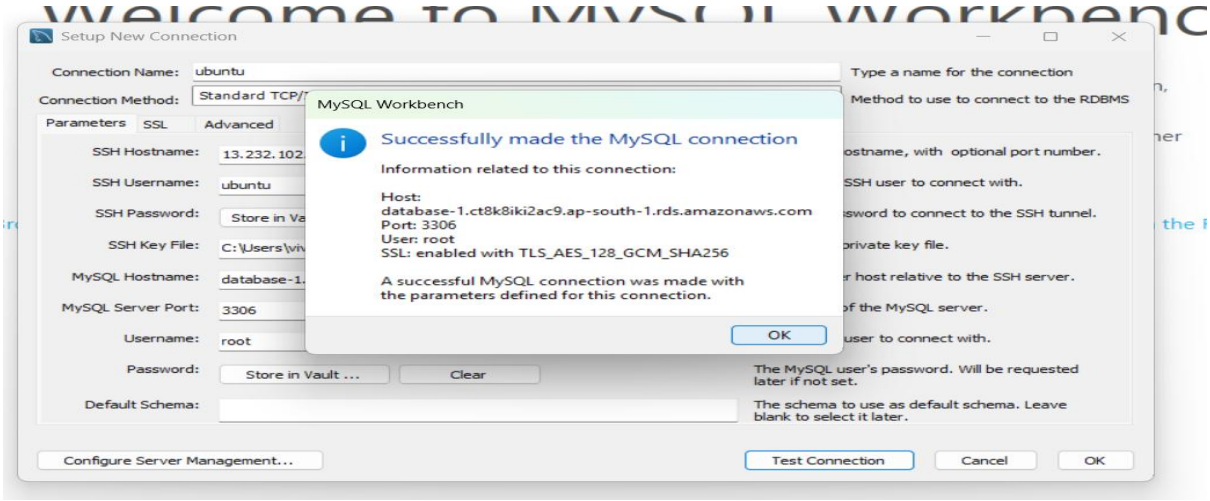


Step 4: Connect to RDS using MySQL Workbench (via SSH)

1. Download and open **MySQL Workbench**,Click **+** to create a new connection
2. Choose:**Connection Method**: Standard TCP/IP over SSH
3. Fill in:
    - **SSH Hostname**: BastionPublicIP (3.110.62.88)
    - **SSH Username**: ubuntu (or ec2-user for Amazon Linux)
    - **SSH Key File**: Select your .pem file
    - **MySQL Hostname**: Private IP of RDS
    - **MySQL Port**: 3306
    - **MySQL Username**: admin
    - **Password**: store password
4. Click **Test Connection** — should be successful!
5. Check the connection in ec2 using telnet cmd

Cmd:[telnet database-1.ct8k8iki2ac9.ap-south-1.rds.amazonaws.com 3306]

```
ubuntu@ip-10-0-1-147:~$ telnet database-1.ct8k8iki2ac9.ap-south-1.rds.amazonaws.com 3306
Trying 10.0.0.227...
Connected to database-1.ct8k8iki2ac9.ap-south-1.rds.amazonaws.com.
Escape character is '^]'.
```

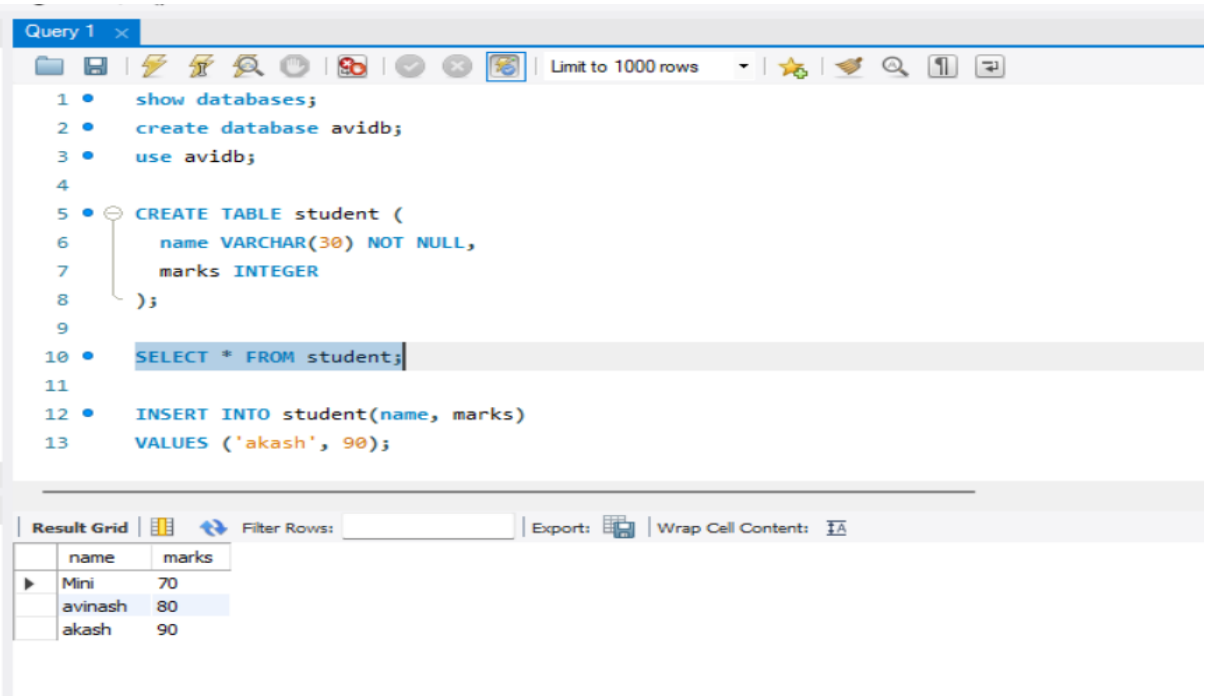


Step 5: Allow DELETE (Safe Updates)

- Go to **Workbench > Edit > Preferences > SQL Editor**,Uncheck **“Safe Updates”**

  Click **OK**,Reconnect to the DB

Step 6: Run SQL Queries

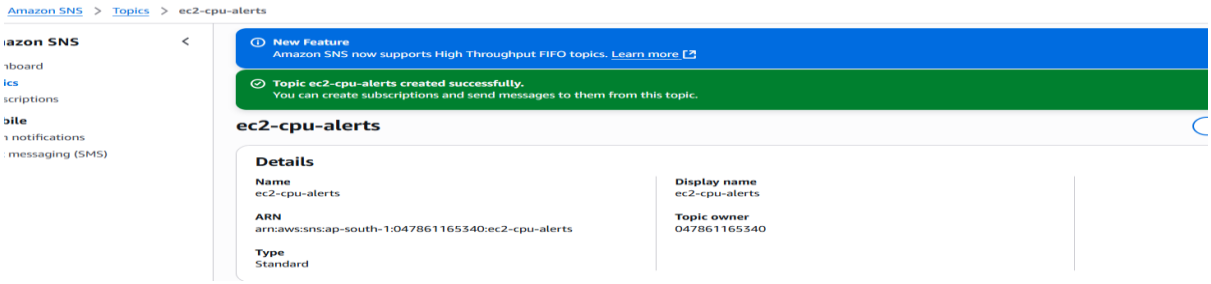


Task 17: Create a cloudwatch alarm to monitor ec2 on CPU utilization and integrate with SNS and perform an EC2 action(stop/terminate).

Task 17.1: Create a custom monitoring for Disk/Ram using CWAgent: [Send memory metrics from EC2 instance to CloudWatch | AWS re:Post]

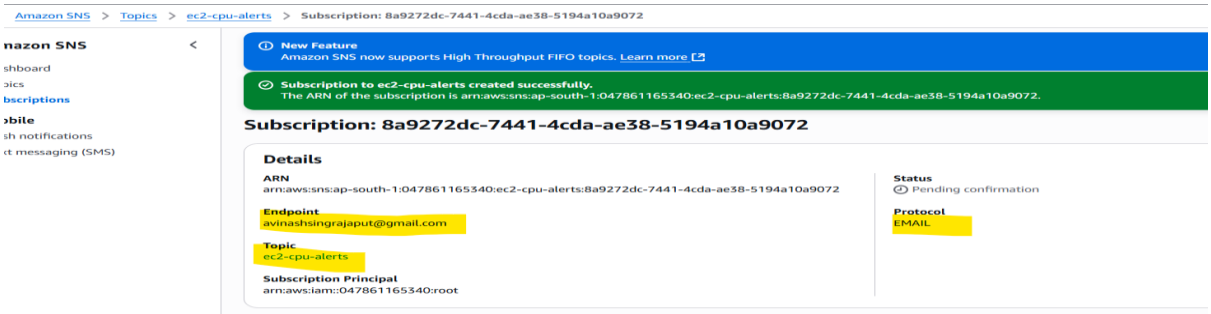
Step 1: Create an SNS Topic

1. Go to AWS Console → **Simple Notification Service (SNS)**,Click **"Create topic"**.
2. Type: Standard,Name: ec2-cpu-alerts,Click **"Create topic"**.



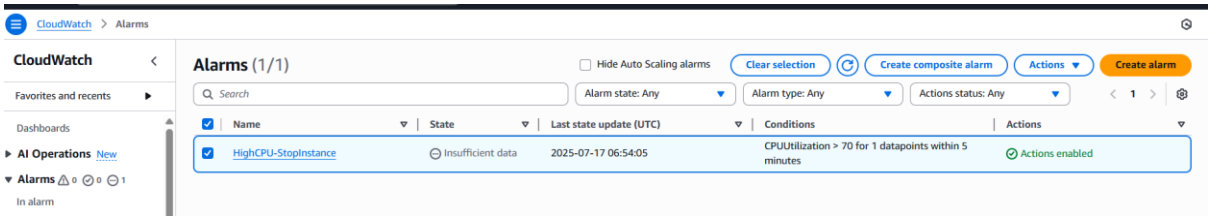
Step 2: Subscribe Your Email

1. In the SNS topic → Click **"Create subscription"**,Protocol: Email,Endpoint: your email address,Confirm the email subscription via your inbox.



Step 3: Create CloudWatch Alarm

1. Go to **CloudWatch** → **Alarms** → **Create Alarm**Select **"EC2"** → **Per-Instance Metrics** → choose your instance → click on CPUUtilization.
2. Statistic: Average, Period: 5 minutes.
3. Condition:Threshold type: Static,Whenever CPU > 70% for 1 datapoint.
4. Actions:Alarm state: Send notification to **SNS topic**,Additional action: **Stop** or **Terminate EC2** (choose as per requirement).
5. Name: High-CPU-Alarm,Click **Create alarm**.



Custom Monitoring for Disk/RAM with CloudWatch Agent

Step 1: Install CloudWatch Agent On your EC2 instance (Ubuntu):

sudo apt update

sudo apt install wget unzip -y

wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb

sudo dpkg -i amazon-cloudwatch-agent.deb

Step 2: Create CloudWatch Agent Config Use the wizard:

```
ubuntu@ip-10-0-1-143:~$ ls
ubuntu@ip-10-0-1-143:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c ssm:AmazonCloudWatch-linux \
-s
****** processing amazon-cloudwatch-agent ******
2025/07/17 06:59:05 E! Fail to fetch/remove json config: ParameterNotFound:
ubuntu@ip-10-0-1-143:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
=============================================================
= Welcome to the Amazon CloudWatch Agent Configuration Manager =
=                                                           =
= CloudWatch Agent allows you to collect metrics and logs from =
= your host and send them to CloudWatch. Additional CloudWatch =
= charges may apply.                                        =
=============================================================
On which OS are you planning to use the agent?
1. linux
2. windows
3. darwin
default choice: [1]:
1
Trying to fetch the default region based on ec2 metadata...
```

- Select EC2, Metrics to collect: **memory, disk**,Destination: CloudWatch. Accept default paths and names,Save the config as /opt/aws/amazon-cloudwatch-agent/bin/config.json

Step 3: Start the Agent :cmd

sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \

-a fetch-config \

-m ec2 \

-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \

-s

```
ubuntu@ip-10-0-1-143:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c ssm:AmazonCloudWatch-linux \
-s
****** processing amazon-cloudwatch-agent ******
I! Trying to detect region from ec2 D! [EC2] Found active network interface I! imds retry client wi
d saved in /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/ssm_AmazonCloudWatch-linu
Start configuration validation...
2025/07/17 07:10:11 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-clou
2025/07/17 07:10:11 I! Valid Json input schema.
2025/07/17 07:10:11 D! ec2tagger processor required because append_dimensions is set
2025/07/17 07:10:11 Configuration validation first phase succeeded
I! Detecting run_as_user...
I! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cl
Configuration validation second phase succeeded
Configuration validation succeeded
ubuntu@ip-10-0-1-143:~$
```

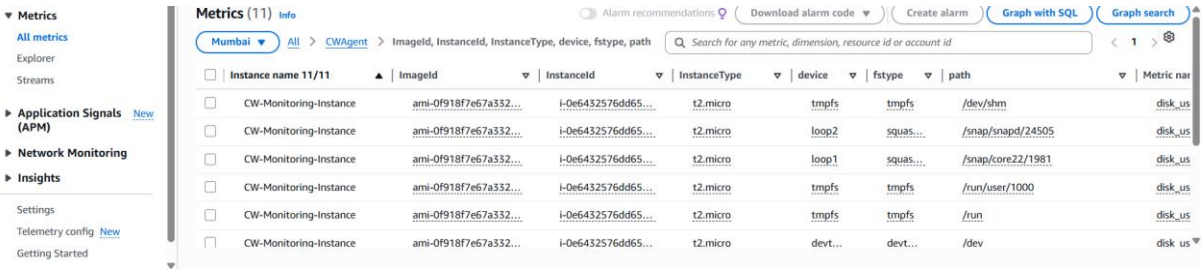
i-0e6432576dd657ec5 (CW-Monitoring-Instance)

PublicIPs: 3.6.41.122 PrivateIPs: 10.0.1.143

CloudShell Feedback

Step 4: View Metrics in CloudWatch

Go to CloudWatch → Metrics → CWAgent namespace → You'll see:
Create alarms if needed, just like the CPU alarm above.



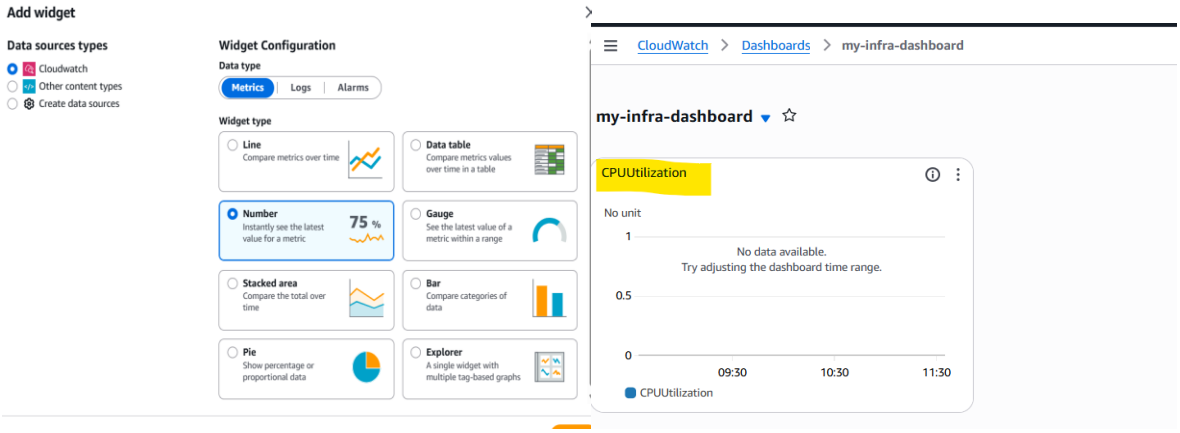
Task 18: Create a cloudwatch Dashboard to monitor all your resources in a single widget window.

Step 1: Open CloudWatch Dashboard Console

1. Go to the **AWS Console**,Navigate to **CloudWatch**,In the left menu → Click on **Dashboards**,Click **Create dashboard**,Enter a name: e.g., my-infra-dashboard Choose **Line**, **Number**, or **Stacked area** widget depending on your preference Click **Next.**

Step 2: Add EC2 CPU Utilization Widget

1. In "Select metric", choose:**Browse → EC2 → Per-Instance Metrics**,Select your instance → Choose CPUUtilization
2. Click **Create widget**



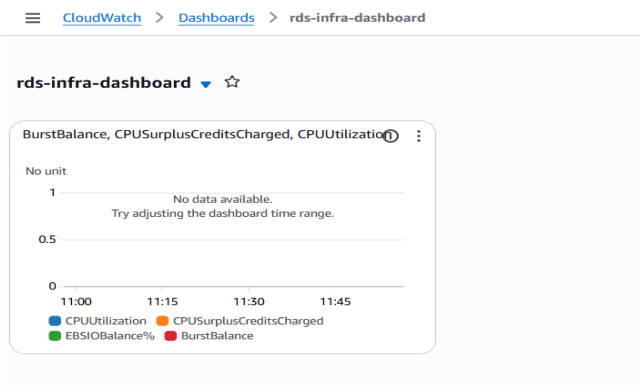
Step 3: Add EC2 Memory or Disk Metrics (Optional but useful)

You need **CloudWatch Agent** installed on EC2 to see memory/disk

1. **Browse → CWAgent → InstanceId**,Select metrics like:mem_used_percent ,disk_used_percent -> Click **Create widget**

Step 4: Add RDS CPU Utilization

1. **Browse → RDS → Per-Database Metrics**,Select your DB → Choose CPUUtilization,Click **Create widget**



Step 6: Finish Dashboard

- Once all widgets are added → Click **Save dashboard**

Task 19: Create a simple Lambda function and test the output.
[Create your first Lambda function - AWS Lambda]

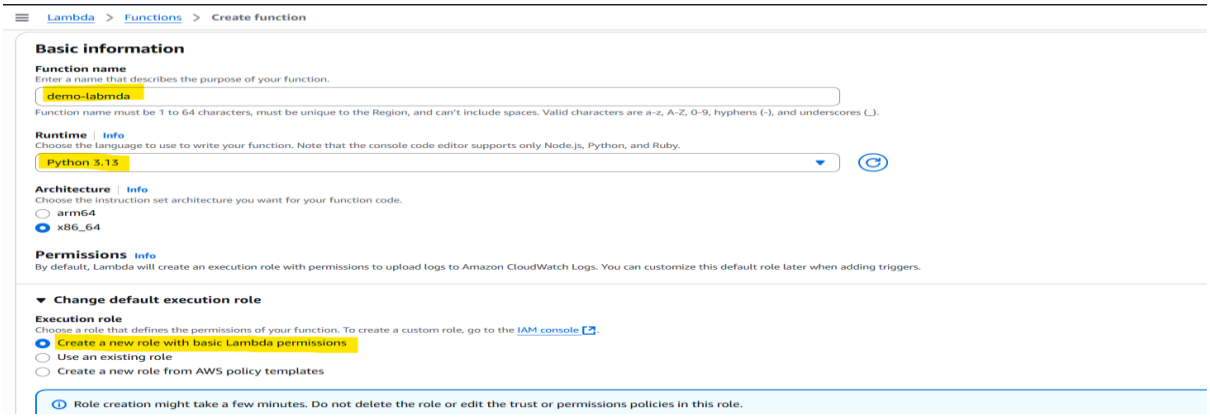
Step 1: Login to AWS Console

Step 2: Navigate to AWS Lambda

1. In the AWS Console, search **"Lambda"** in the search bar and click on it.
2. Click on **“Create function”**.

Step 3: Create the Lambda Function

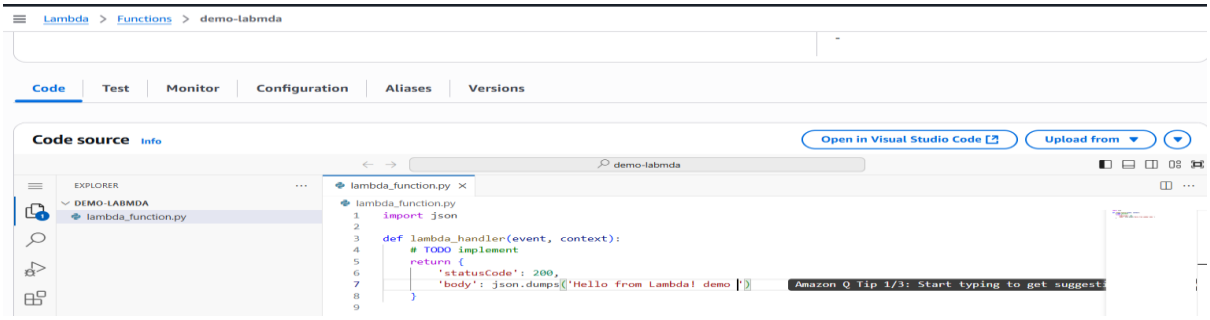
1. **Choose “Author from scratch”**
2. Fill the form:**Function name**: HelloLambdaFunction,**Runtime**: Select Python 3.12 (or Node.js, depending on your preference),**Permissions**:Leave it as **“Create a new role with basic Lambda permissions”**
3. Click **Create function**



Step 4: Write Your Code

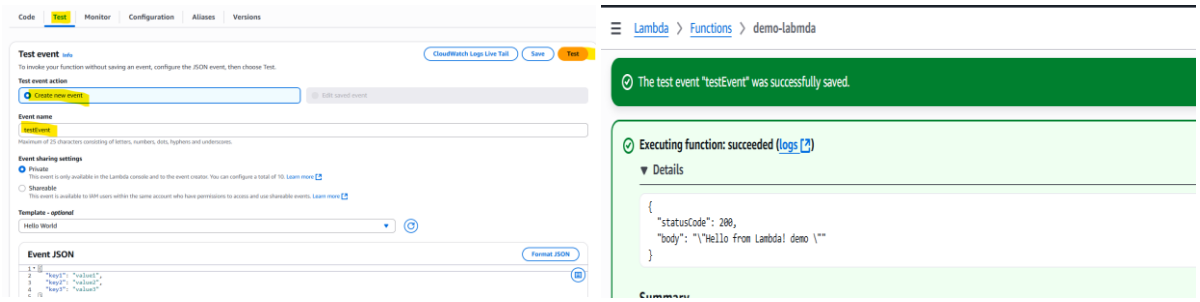
```
def lambda_handler(event, context):
  return {
    'statusCode': 200,
    'body': 'Hello from Lambda!'
  }
```

Click Deploy (top-right corner).



Step 5: Test the Lambda Function

1. Click on Test (right of Deploy).,Create a test event:Event name: testEvent,Leave the default JSON as it is: Click **Save** then **Test**



Step 6: Check Output

- You will see the **execution results** below:Status: **"Execution result: succeeded"**.

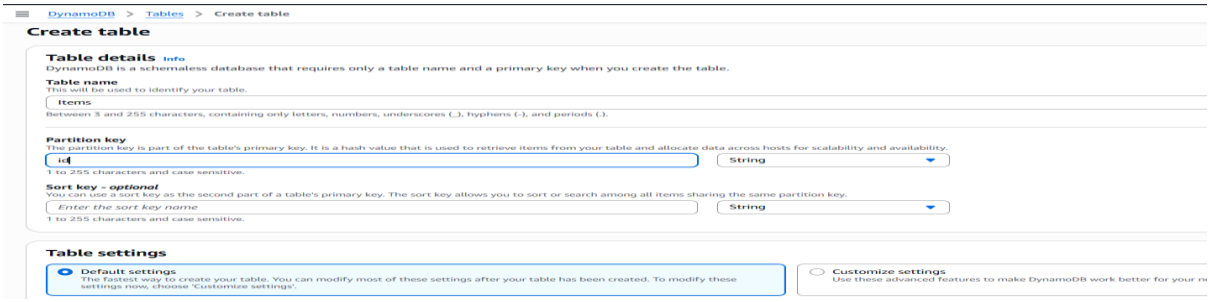
Task 20: Create a CRUD HTTP API with Lambda and DynamoDB - Amazon API Gateway by following the link below

https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-dynamo-db.html

---

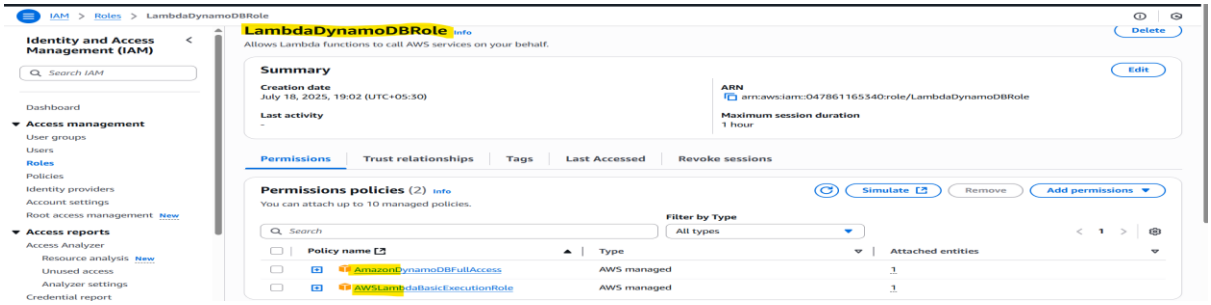
Step 1: Create a DynamoDB Table

1. Go to **AWS Console > DynamoDB**
2. Click **“Create Table”**:Table name: Items,Partition key: id (String)
3. Leave all other settings as default and click **Create table**



Step 2: Create IAM Role for Lambda

1. Go to **IAM > Roles** → Click **Create Role** Select **Lambda** as the trusted service
2. Attach these policies:
    - AWSLambdaBasicExecutionRole
    - AmazonDynamoDBFullAccess *(or use least privilege if preferred)*
3. Name the role: LambdaDynamoDBRole-Create the role.

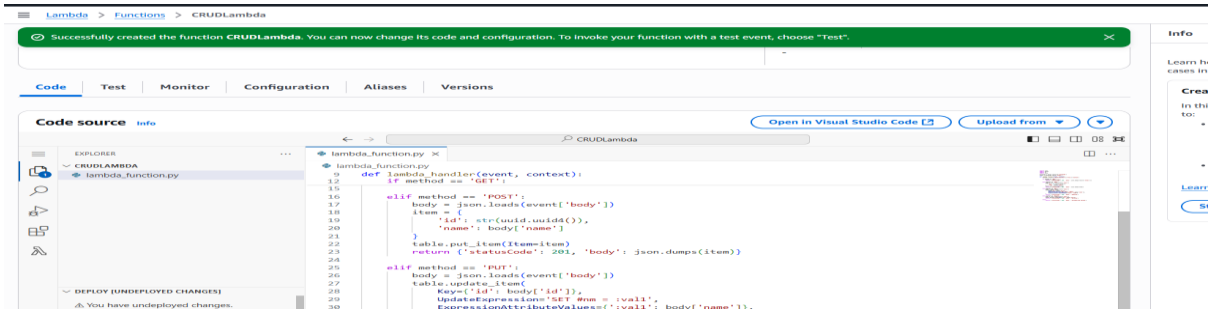


Step 3: Create a Lambda Function

1. Go to **AWS Lambda**
2. Click **Create function**,Name: CRUDLambda,Runtime: Python 3.9 or Node.js (choose your preference),Permissions: Use existing role → select LambdaDynamoDBRole, Click **Create function**

Step 4: Add Code to Lambda Function

Example: Python Lambda (simple CRUD) Paste this in the **Lambda function code editor**:



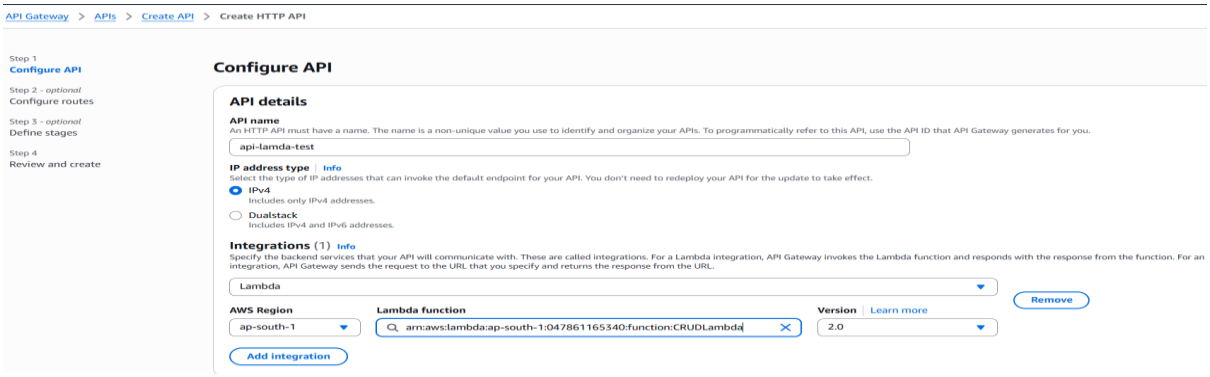
Lambda function code:

```
import json
import boto3
import os
import uuid
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Items')
def lambda_handler(event, context):
  method = event['requestContext']['http']['method']
  if method == 'GET':
    result = table.scan()
    return {'statusCode': 200, 'body': json.dumps(result['Items'])}
  elif method == 'POST':
    body = json.loads(event['body'])
    item = {
      'id': str(uuid.uuid4()),
      'name': body['name']
    }
    table.put_item(Item=item)
    return {'statusCode': 201, 'body': json.dumps(item)}
  elif method == 'PUT':
    body = json.loads(event['body'])
    table.update_item(
      Key={'id': body['id']},
      UpdateExpression='SET #nm = :val1',
      ExpressionAttributeValues={':val1': body['name']},
      ExpressionAttributeNames={'#nm': 'name'}
    )
    return {'statusCode': 200, 'body': 'Updated'}
  elif method == 'DELETE':
    body = json.loads(event['body'])
```

```
        table.delete_item(Key={'id': body['id']})
        return {'statusCode': 200, 'body': 'Deleted'}
    else:
        return {'statusCode': 400, 'body': 'Unsupported method'}
```

Step 5: Create HTTP API via API Gateway

1. Go to **Amazon API Gateway**,Choose **HTTP API**,Click **Build**

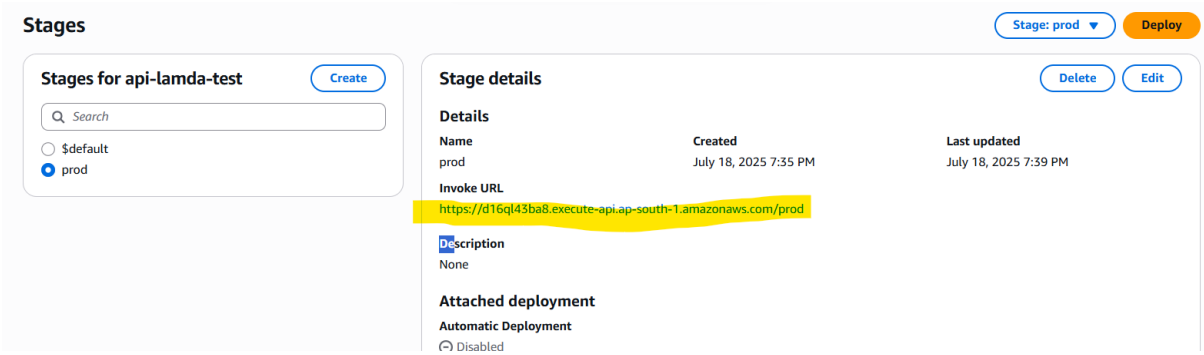


Step 6: Configure API

1. **Choose Integration**:Integration type: **Lambda**,Select your Lambda: CRUDLambda,Click **Next**
2. **Configure routes**:Add route: ANY /items,Attach to the same Lambda function
3. Click **Next**

Step 7: Deploy API

1. Create a stage: prod,Click **Next**, then **Create**,Copy the **Invoke URL** (you'll use it to test)



✅ Step 8: Test Your API

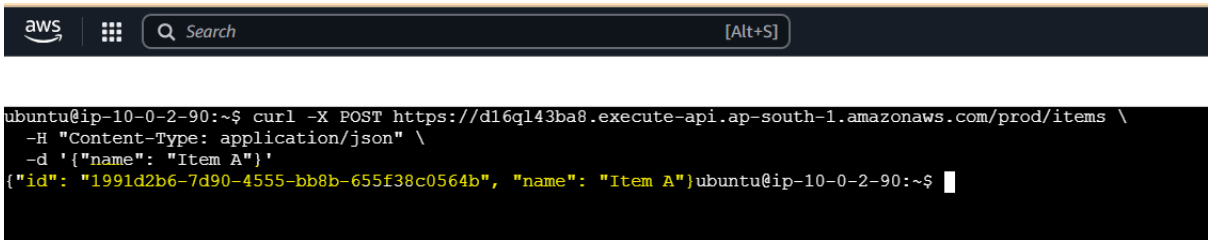
You can use **Postman** or **curl**.

POST - Create Item:

curl -X POST https://d16ql43ba8.execute-api.ap-south-1.amazonaws.com/prod/items \

-H "Content-Type: application/json" \

-d '{"name": "Item A"}'



GET - List Items:

curl -X GET https://d16ql43ba8.execute-api.ap-south-1.amazonaws.com/prod/items

3. PUT (Update an Item)

curl -X DELETE https://d16ql43ba8.execute-api.ap-south-1.amazonaws.com/prod/items/1

4. DELETE (Delete an Item)

curl -X DELETE https://d16ql43ba8.execute-api.ap-south-1.amazonaws.com/prod/items/1

CAPSTONE PROJECT: Build a solution for a 3 Tier web application.

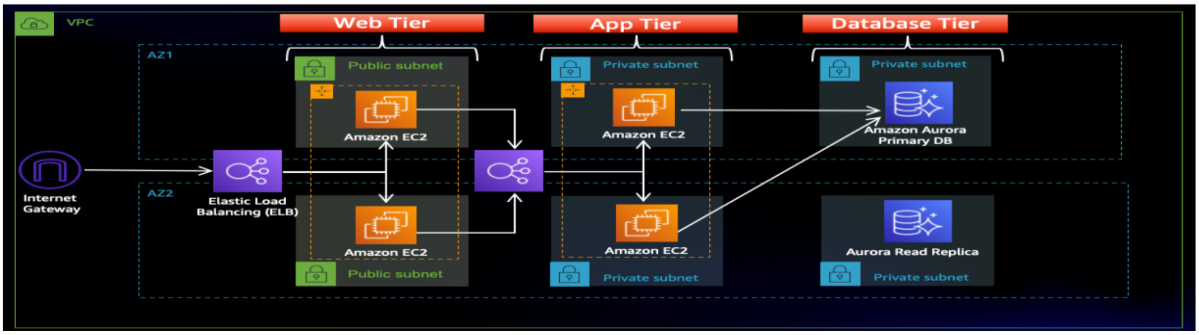
Build a custom VPC with 4 subnets 2 private and 2 public

R53 --> ALB --> TG --> NGINX(Public Subnet) --> APP(Private Subnet)

Application should perform CRUD on the RDS. Configure Cloudwatch metrics on the ec2 instances and create dashboards based on app

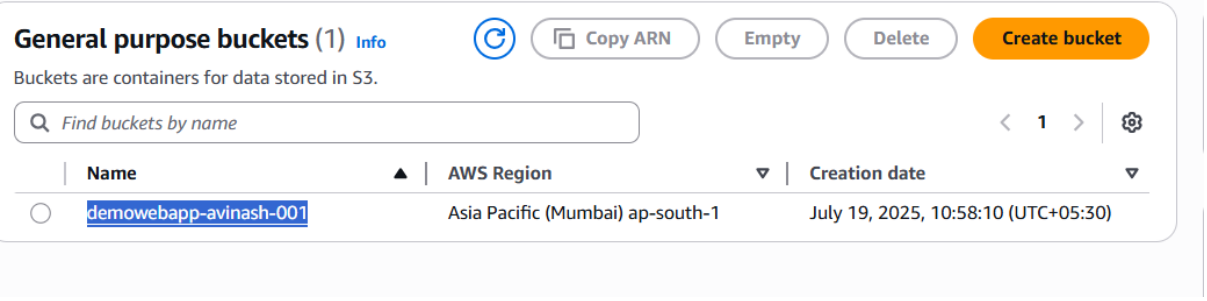
AWS BACKUP should be implemented

Architecture Overview



1st clone the repo and create the s3 bucket and IAM role

- S3 bucket name: demowebapp-avinash-001
- IAM roles: 'AmazonSSMManagedInstanceCore', 'AmazonS3ReadOnlyAccess'
- Repo: git clone https://github.com/aws-samples/aws-three-tier-web-architecture-workshop.git



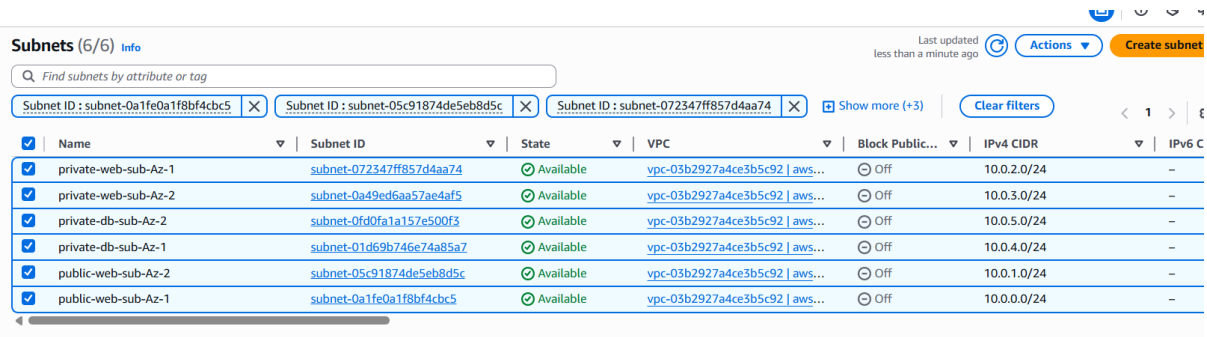
1. Create a Custom VPC:VPC Configuration

- **CIDR Block**: 10.0.0.0/16
- Enable DNS hostnames and resolution
- **Create subnets:**

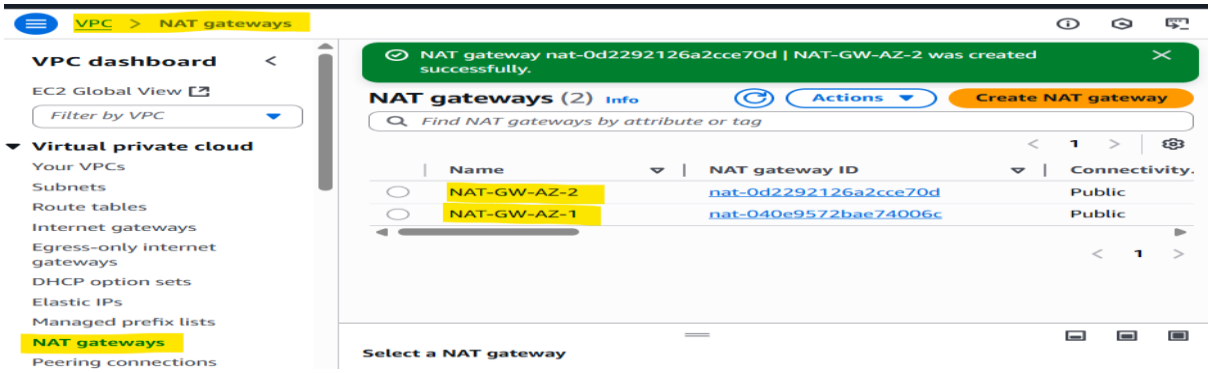
  2 Public Subnets (e.g., 10.0.1.0/24, 10.0.2.0/24)

  2 Private Subnets (e.g., 10.0.3.0/24, 10.0.4.0/24)

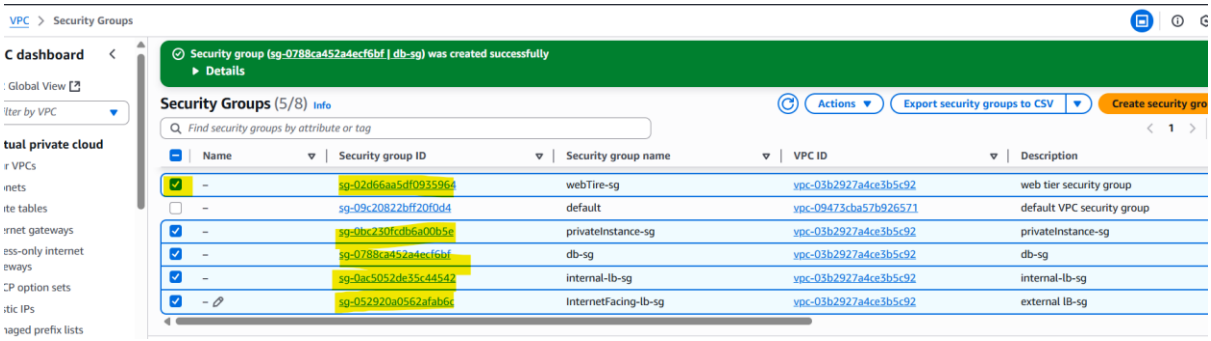
  Deploy across 2 Availability Zones for high availability



- Internet Gateway , NAT gateway and Route Tables



- Create a security group



2. Launch EC2 Instances

NGINX Instance (Frontend) Launch in **public subnet,**Use Ubuntu/AL2,Install NGINX

sudo apt update

sudo apt install nginx -y

App Server (Backend)

- **Launch in private subnet,Use Ubuntu/AL2**
- **Setup your app to connect to RDS (NodeJS, Flask, etc.)**

Connect ec2 then run these cmds

Run the cmds :

sudo apt update && sudo apt upgrade -y

sudo apt install mysql-client -y

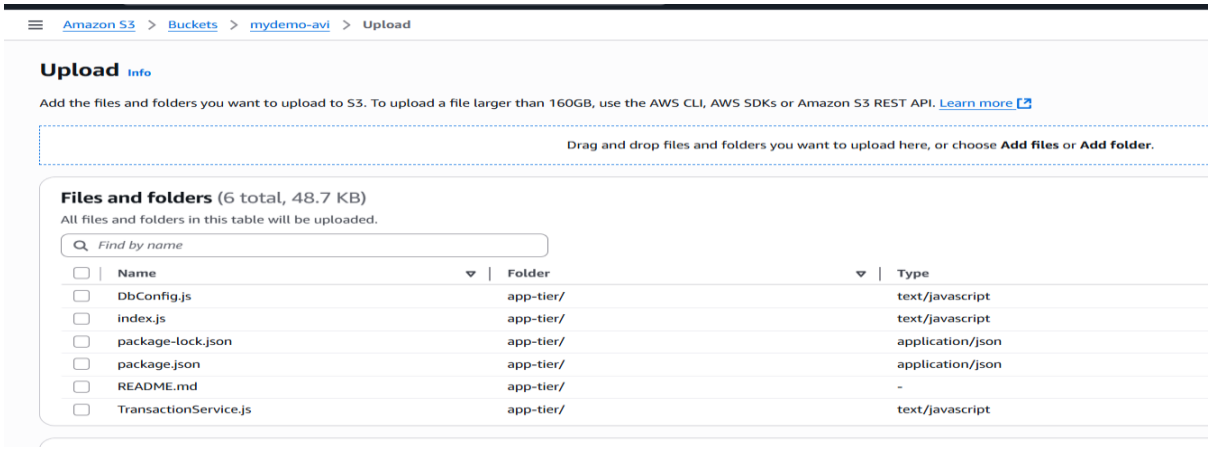
mysql –version

mysql -h database-1-instance-1.ct8k8iki2ac9.ap-south-1.rds.amazonaws.com -u admin -p

```
Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create DATABASE webappdb;
Query OK, 1 row affected (0.01 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| webappdb           |
+--------------------+
5 rows in set (0.00 sec)

mysql> use webappdb;
Database changed
```

```
mysql> show tables;
+--------------------+
| Tables_in_webappdb |
+--------------------+
| transactions       |
+--------------------+
1 row in set (0.02 sec)

mysql> INSERT INTO transactions (amount,description) VALUES ('400','groceries');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM transactions;
+----+--------+-------------+
| id | amount | description |
+----+--------+-------------+
|  1 | 400.00 | groceries   |
+----+--------+-------------+
1 row in set (0.00 sec)
```

Upload the app file In s3 buckets



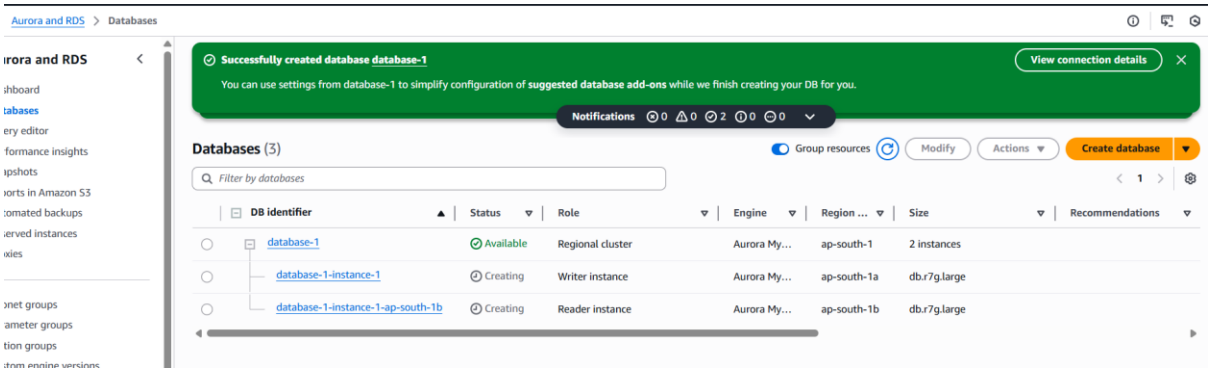
3. Application Logic (CRUD with RDS)

Create RDS (MySQL/PostgreSQL)

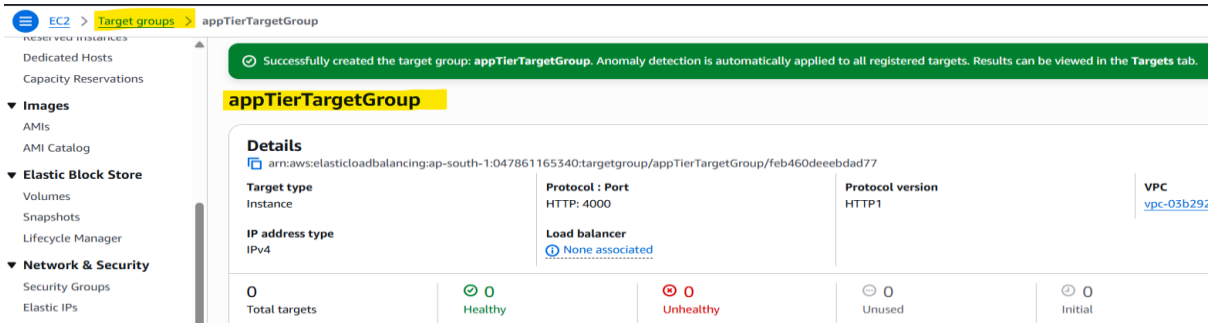
- **Deploy in private subnets**
- **Enable public access = No**
- **Enable IAM DB authentication (optional)**
- **Configure security group to allow DB access only from App server SG**

Connect App with RDS

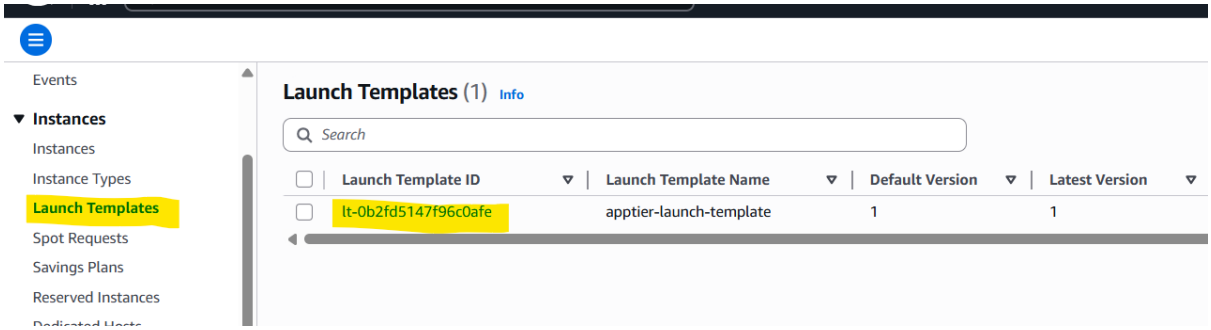
- **Set DB host in environment variables**
- **Build API with basic CRUD:**
    - **Create, Read, Update, Delete**



New create a target group

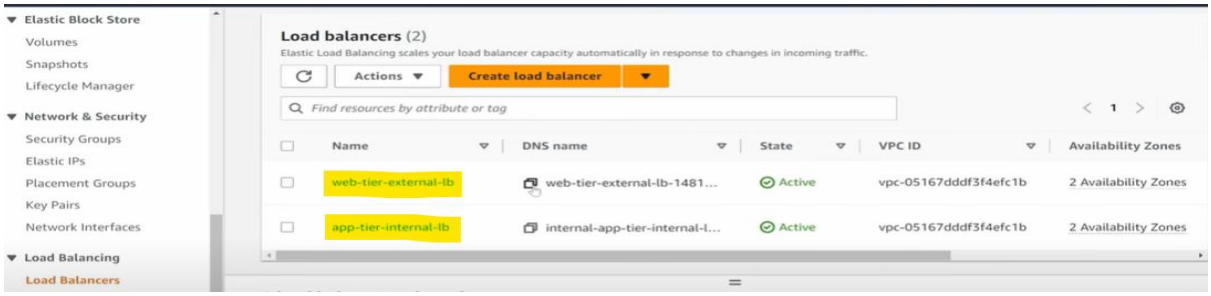


Then we need to create launch template

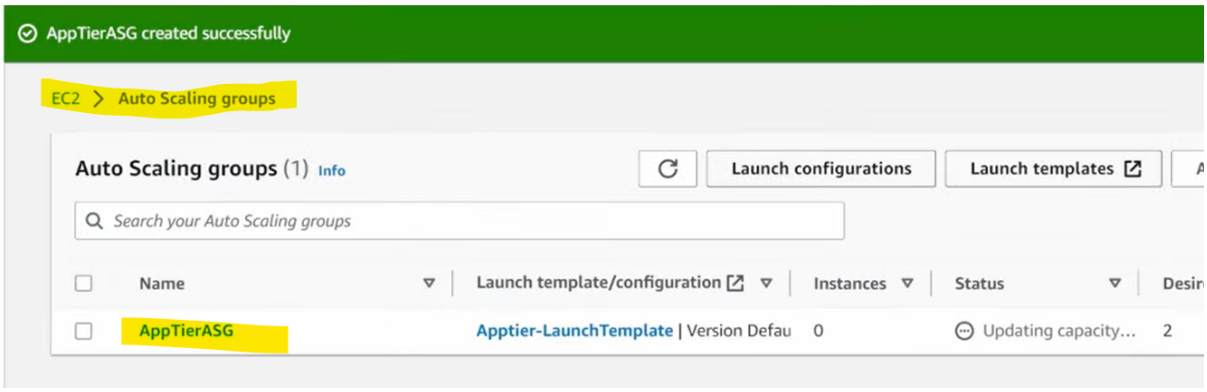


4. Application Load Balancer

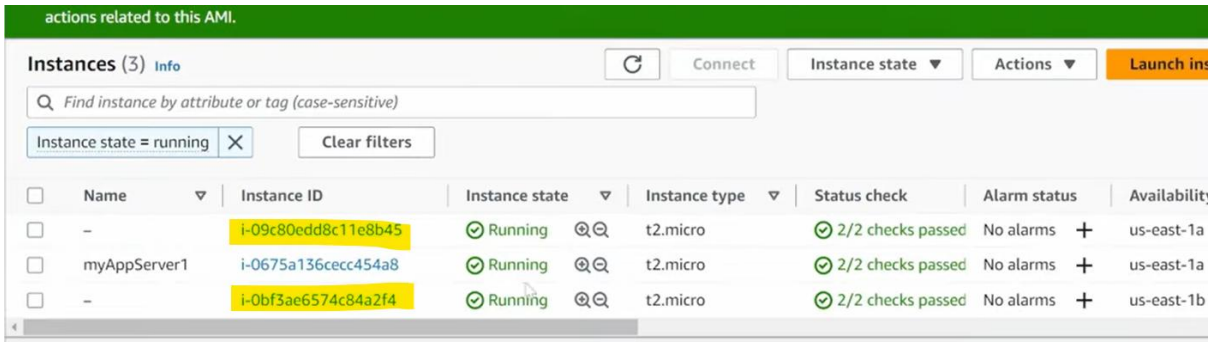
- **Create ALB in public subnets**
- **Create Target Group for NGINX instance (use instance or IP)**
- **Attach ALB listener (HTTP:80) forwarding to TG**
- **Ensure NGINX SG allows ALB SG**



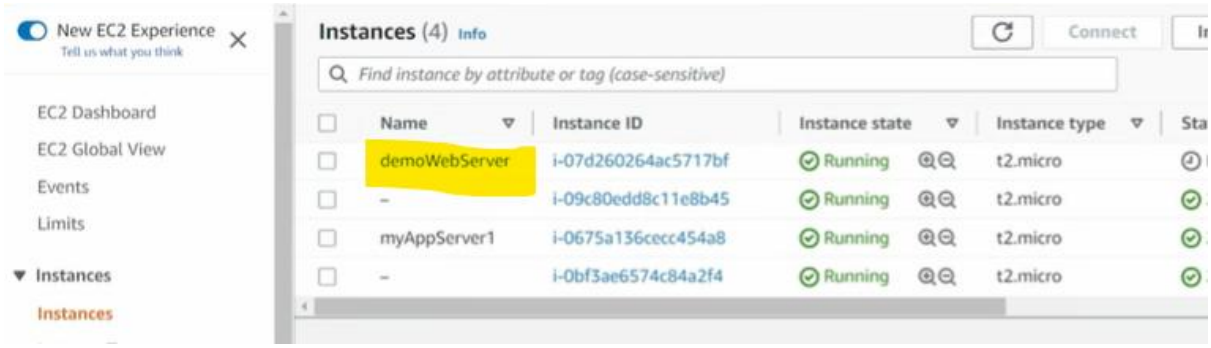
Create auto scaling groups



After created the Auto scaling group 2 instance are running



Now create a new instance then start the ec2 demoWebServer



Then I need to update the nginx.conf dir ,there I need to add the DNS name of load balancer

[internal-app-tier-internal-Lb-1716221266.ap-south-1.elb.amazonaws.com]

```
        #react app and front end files
        location / {
        root    /home/ec2-user/web-tier/build;
        index index.html index.htm
        try_files $uri /index.html;
        }

        #proxy for internal lb
        location /api/{
                proxy_pass http://internal-app-tier-internal-lb-677405433.us-east-1.elb.amaz
onaws.com]:80/;
        }
```

Run the nginx it must in active state: sudo service nginx status

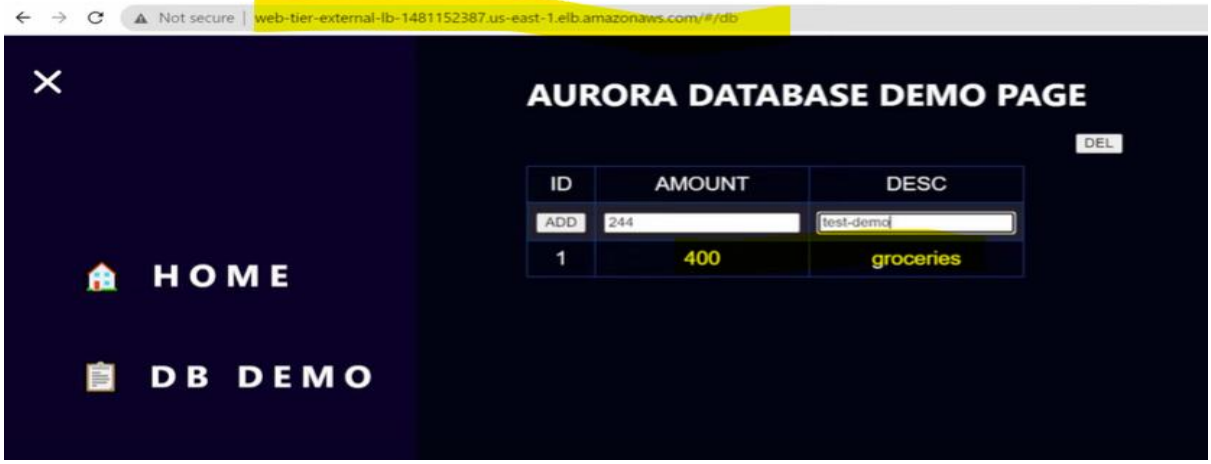
Give permissions: chmod -R 755 /home/ec2-user

```
0c2539f385b1a6d94          07d260264ac5717bf
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: d
     Active: active (running) since Sat 2023-06-17 02:49:36 UTC; 9s ago
    Process: 26218 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, sta
    Process: 26226 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCE
    Process: 26232 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 26240 (nginx)
      Tasks: 2 (limit: 1114)
     Memory: 2.2M
        CPU: 56ms
     CGroup: /system.slice/nginx.service
             ├─26240 "nginx: master process /usr/sbin/nginx"
             └─26242 "nginx: worker process"
```

Final Testing Access domain via browser :

[web-tier-external-lb-1481152387.us-east-1.elb.amazonaws.com/#/db]

- **Perform CRUD operations**
- **Validate CloudWatch dashboards**
- **Trigger a manual backup via AWS Backup**



Resource	Inbound	Outbound
NGINX EC2	HTTP (80) from ALB	All
App EC2	Custom App Port from NGINX	RDS Port
RDS	DB Port (3306/5432) from App SG	N/A
ALB	HTTP (80) from Internet	NGINX Target